



May 2024

Constraint Programming: an introduction

Dr Ing. Houndji V. Ratheil

<https://ratheil.info>

Lecturer-Researcher at UAC



Houndji Vinasetan Ratheil

<https://ratheil.info>

Teaching experience (since my PhD thesis)

- Artificial Intelligence since 2014
- Optimisation since 2015
- Object Oriented Modelling/Programming
- Reliability of Information System
- etc.



Head of Software Engineering department at IFRI, UAC

Outline

General Introduction

Global Constraints

How CP works concretely ?

Take Away Message

General Introduction



Constraint Programming (CP) is a paradigm derived from artificial intelligence, operational research, and algorithmic that can be used to solve combinatorial optimization problems.

CP solves problems by interleaving search (assigning a value to an unassigned variable) and propagation (removing inconsistent values).



General Introduction



Constraint Programming allows a declarative description of the problem to solve.

General Introduction



Constraint Programming allows a declarative description of the problem to solve.

Other declarative approaches : **Logic Programming, Operations Research**

Find a formal representation of the knowledge with logical rules

$A, B \Rightarrow G$
 $D \Rightarrow A$
 $C \Rightarrow H$
 $I \Rightarrow F$
 $H, F, J \Rightarrow B$
 $H, K \Rightarrow J$
 $C, F \Rightarrow K$

Find a mathematical formulation of the problem

$$\begin{aligned} s_0^i &= 0, \forall i \\ x_t^i + s_{t-1}^i &= d_t^i + s_t^i, \forall i, t \\ x_t^i &\leq y_t^i, \forall i, t \\ \sum_i y_t^i &= 1, \forall t \\ \chi_t^{i,j} &\geq y_{t-1}^i + y_t^j - 1, \forall i, j, t \\ x_t^i, y_t^i, \chi_t^{i,j} &\in \{0, 1\}, s_t^i \in \mathbb{N}, \forall i, j \in [1, \dots, m] \end{aligned}$$

General Introduction



In CP we would like to describe the problem in natural “english” language.

General Introduction



In CP we would like to describe the problem in natural “english” language.

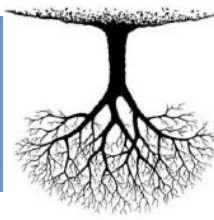
In practice it looks like

```
val nQueens = 8 // Number of queens
val Queens = 0 until nQueens

// Variables
val queens = Array.fill(nQueens)(CPIntVar.sparse( minValue = 0, nQueens - 1))

// Constraints
add(allDifferent(queens))
add(allDifferent(Queens.map(i => queens(i) + i)))
add(allDifferent(Queens.map(i => queens(i) - i)))
```


CP Applications



Production Planning



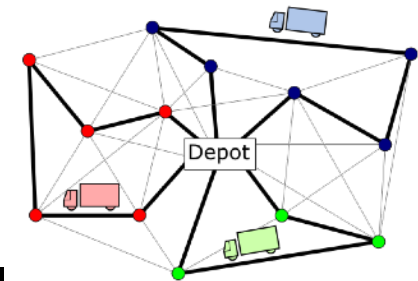
Agricultural land allocation



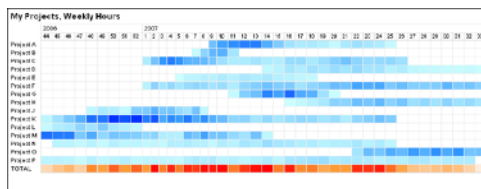
Rail Applications



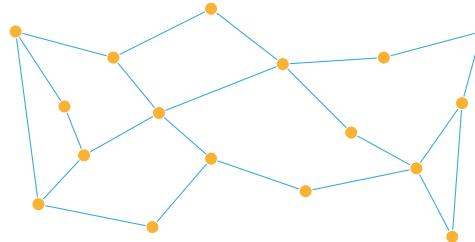
Vehicle Routing



Scheduling



Traffic Engineering



and much more...

A library of problems



A live demo with OscaR

A library of problems



CSPLib: A problem library for constraints

<http://csplib.org/>

058: Discrete Lot Sizing Problem

[Specification](#) [Data files](#) [Results](#) [References](#) [Models](#) [Cite](#) [Json](#)

[Edit Page](#)

Proposed by Vinasetan Ratheil Houndji, Pierre Schaus, Laurence Wolsey, Yves Deville

Discrete Lot Sizing and Scheduling Problem (DLSP) is a production planning problem which consists of determining a minimal cost production schedule (production costs, setup costs, changeover costs, stocking costs, etc.), such that machine capacity restrictions are not violated, and demand for all products is satisfied. The planning horizon is discrete and finite.

The variant described here is the one used for experiments in [The StockingCost Constraint](#).

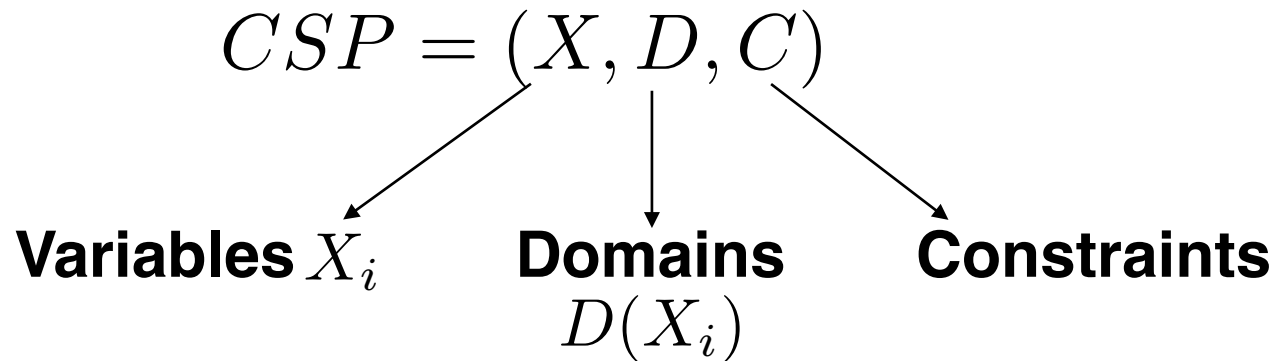
It is a multi-item, single machine problem with capacity of production limited to one per period. There are storage costs and sequence-dependent changeover costs, respecting the triangle inequality. Each order consisting of one unit of a particular item has a due date and must be produced at latest by its due date. The stocking (inventory) cost of an order is proportional to the number of periods between the due date and the production period. The changeover cost $q^{i,j}$ is induced when passing from the production of item i to another one j with $q^{i,i} = 0, \forall i$. Here, backlogging is not allowed. The objective is to assign a production period for each order respecting its due date and the machine capacity constraint so as to minimize the sum of stocking costs and changeover costs.

Example : Consider the problem with the following input data: number of items type $nbItems = 2$; number of periods $nbPeriods = 5$; stocking cost $h = 2$; demand times for items of type 1 $d_{i \in 1, \dots, 5}^1 = (0, 1, 0, 0, 1)$ and for items of type 2 $d_{i \in 1, \dots, 5}^2 = (1, 0, 0, 0, 1)$; $q^{1,2} = 5, q^{2,1} = 3$. A feasible solution of this problem is $productionPlan = (2, 1, 2, 0, 1)$ which means that item 2 will be produced in period 1; item 1 in period 2; item 2 in period 3 and item 1 in period 5. Note that there is no production in period 4, it is an idle period. The cost associated to this solution is $q^{2,1} + q^{1,2} + q^{2,1} + 2 * h = 15$ but it is not the optimal cost. The optimal solution is $productionPlan = (2, 1, 0, 1, 2)$ with the cost $q^{2,1} + q^{1,2} + h = 10$.

A [Simulated Annealing metaheuristic approach](#) along with a dataset of [large-size instances](#) has been developed by Sara Ceschia, Luca Di Gaspero and Andrea Schaerf. An online web solution checker and solution repository is available at <https://ophub.uniud.it/problem/lsp>.

This library presents a list of 92 problems
in 14 categories proposed by 70 authors.

General Introduction



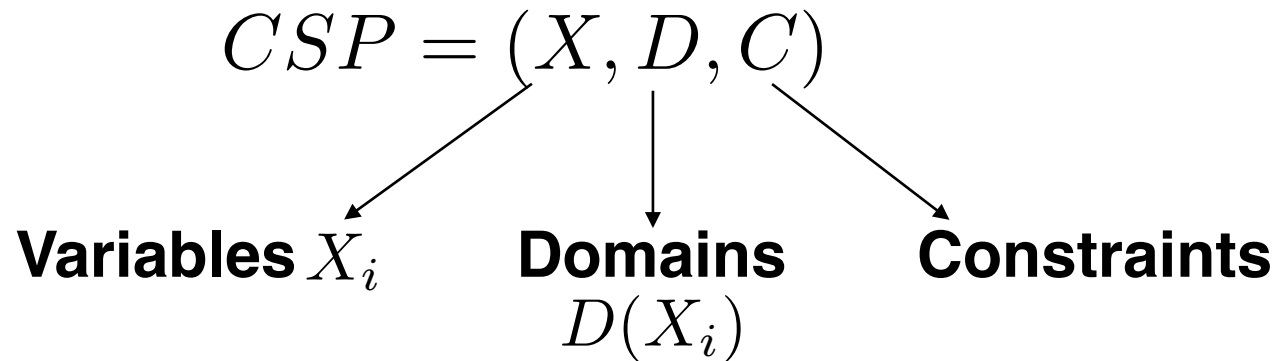
Solution of **CSP**:

- An assignment of all variables
- All constraints are satisfiable

Model
+
Resolution

COP (Constraint Optimisation Problem):
 $CSP + \text{objectif function(s)}$

General Introduction



Example: $X_1 \in \{1, 2\}$ $X_2 \in \{1, 2\}$ $X_1 \neq X_2$

A solution ?

Modeling : an example 1



A farmer sends his son to the market with XOF100.000. With this XOF100.000, the son must buy a hundred animals. He has to come back to the farm with the 100 animals and he has to spend the full XOF100.000. A chick costs XOF500, a pig XOF5.000, and an ox XOF20.000. The son must buy at least one animal of each species.

How many chicks, pigs, and oxen will the son have?

To do : propose a CP model for this problem.

Modeling : an example 2



Coin return problem

We are interested in a beverage vending machine.

The user inserts coins for a total of **T** fcfa, then selects a drink, the price of which is **P** fcfa. It is necessary to calculate the change to be returned, knowing that the distributor has in reserve **E2** coins of 200f, **E1** coins of 100f, **C50** coins of 50f, **C25** coins of 25f, and **C10** coins of 10f.

To do : propose a CP model for this problem.

What should be added if we want to minimize the number of coins to be returned.

Modeling : an example 3



The steelworks

A small steel mill (steel manufacturing plant) must decide on the allocation for the next week (40 hours in total) of its rolling mill (machine used to reduce the thickness of materials). This processes unfinished steel plates and can produce two types of semi-finished products, **strips** and **rolls**. The rolling mill can produce 200 strips or 140 rolls per hour, knowing that a strip brings in \$25 and a roll \$30. Finally, it is not necessary to produce more than the total of the programmed orders, namely 6000 strips and 4000 rolls.

Determine the respective quantities to be produced to maximize profit.

To do : propose a CP model for this problem.

General Introduction



**Model
+
Resolution**

General Introduction



$$CSP = (X, D, C)$$

Variables X_i

Domains
 $D(X_i)$

Constraints

Model
+
Resolution

General Introduction



$$CSP = (X, D, C)$$

Variables X_i

Domains
 $D(X_i)$

Constraints

Example $X_1 \in \{1, 2\}$ $X_2 \in \{1, 2\}$ $X_1 \neq X_2$

Model
+
Resolution

General Introduction



$$CSP = (X, D, C)$$

Variables X_i

Domains
 $D(X_i)$

Constraints

Model
+
Resolution

Filtering + Search

General Introduction



$$CSP = (X, D, C)$$

Variables X_i

Domains
 $D(X_i)$

Constraints

Model
+
Resolution

Filtering + Search

General Introduction

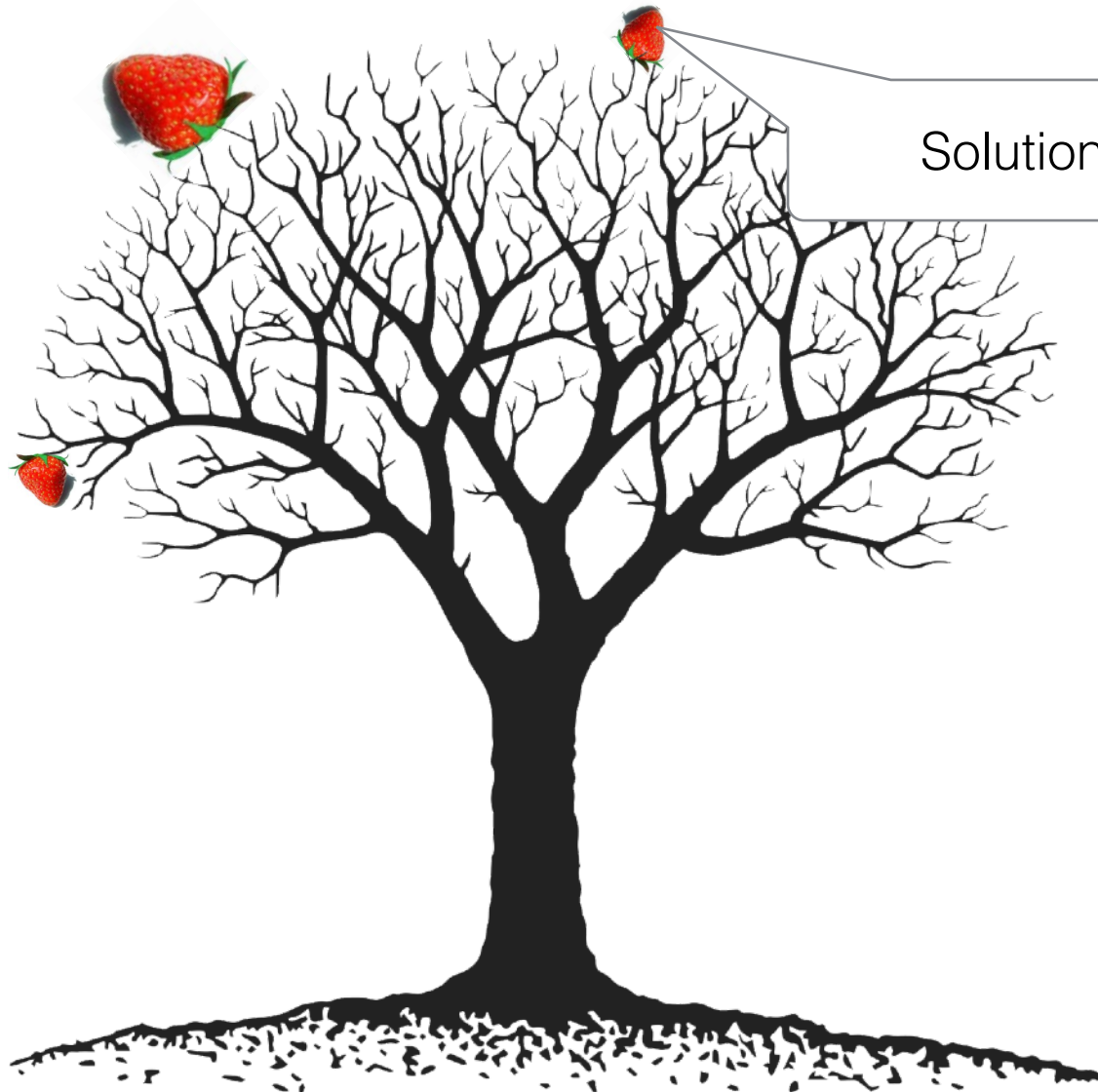


Resolution in CP : an intuition



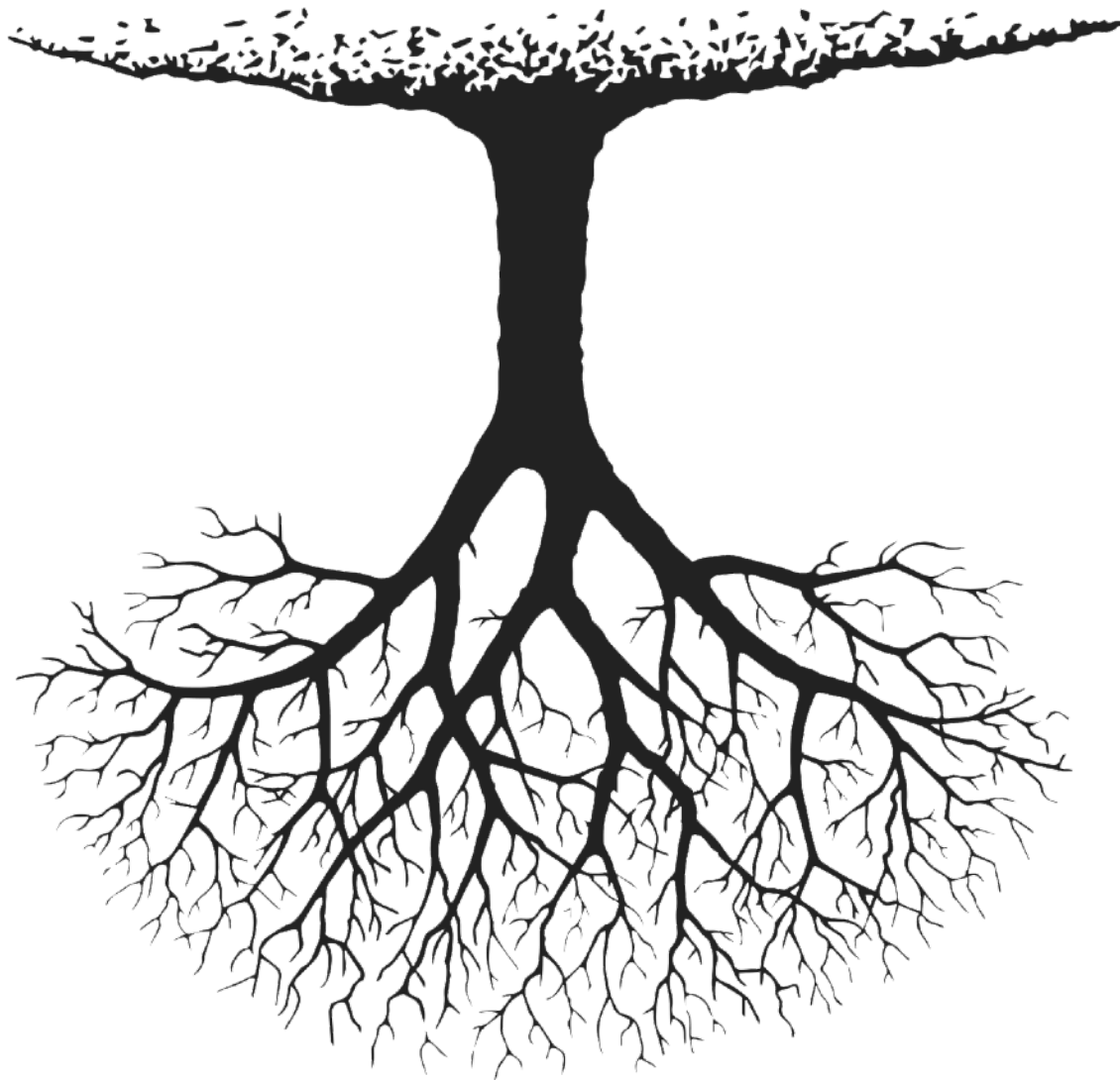
Fruit seeking

General Introduction



Solution = Fruit

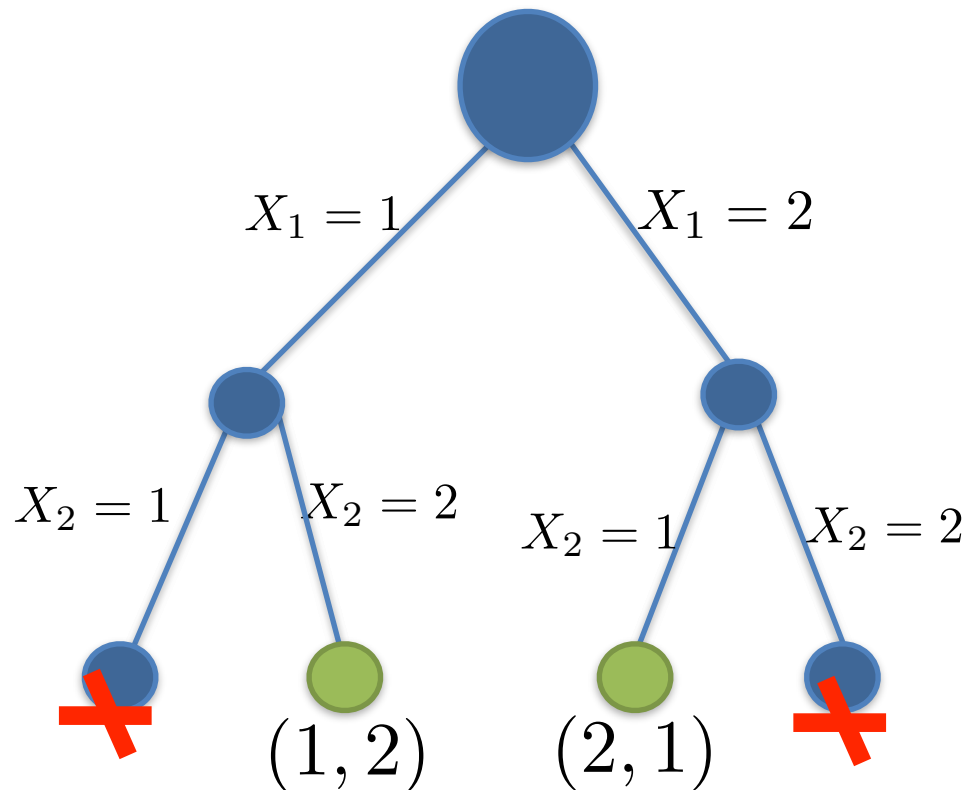
General Introduction



General Introduction



Example: $X_1 \in \{1, 2\}$ $X_2 \in \{1, 2\}$ $X_1 \neq X_2$

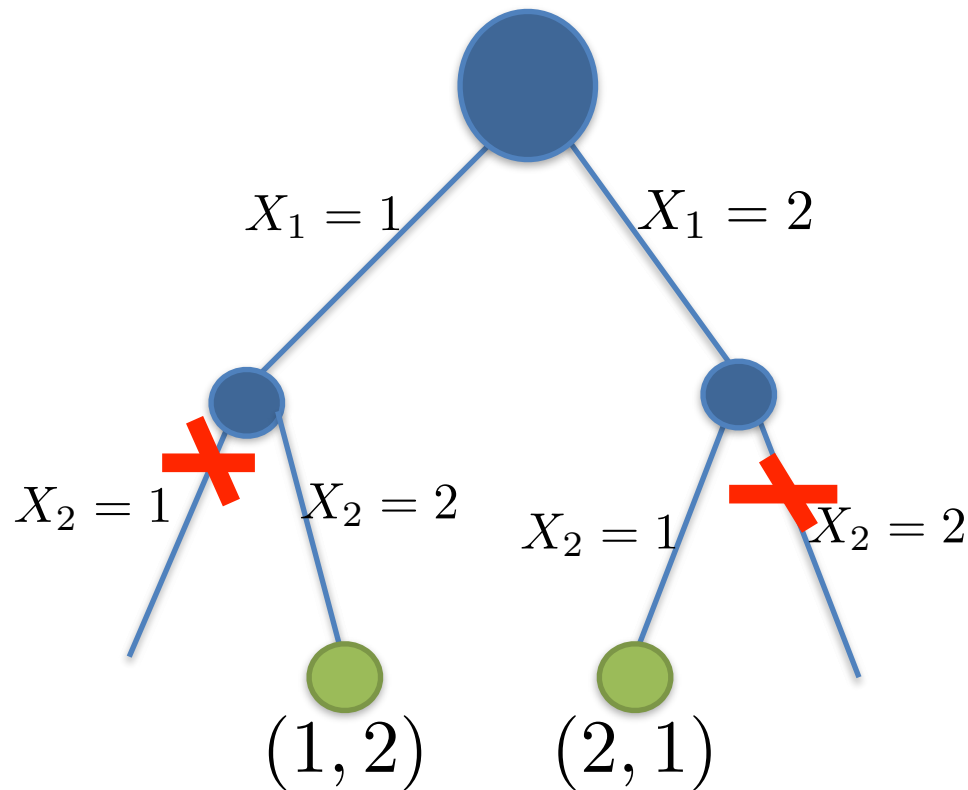


Search

General Introduction



Example : $X_1 \in \{1, 2\}$ $X_2 \in \{1, 2\}$ $X_1 \neq X_2$

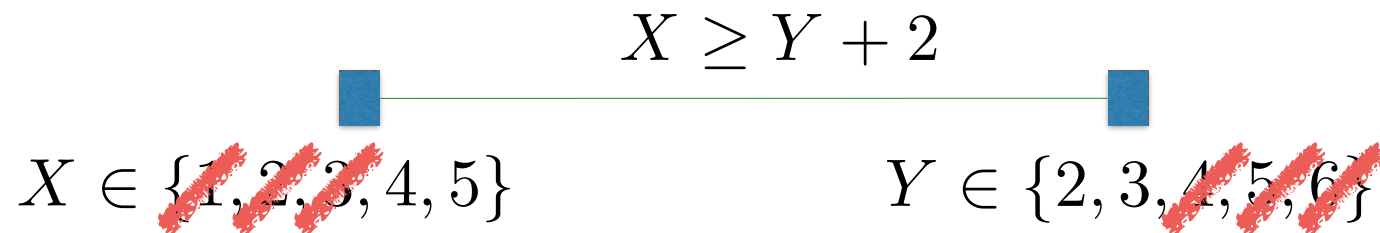


Filtering

General Introduction

Filtering removes inconsistent values

Example 2:



General Introduction



CP = Search + Filtering

Too small



Too big

Outline

General Introduction

Global Constraints

How CP works concretely ?

Take Away Message

Global Constraints



Global constraint :

- A constraint reasoning on many variables at the same time
- Specialized, powerful filtering

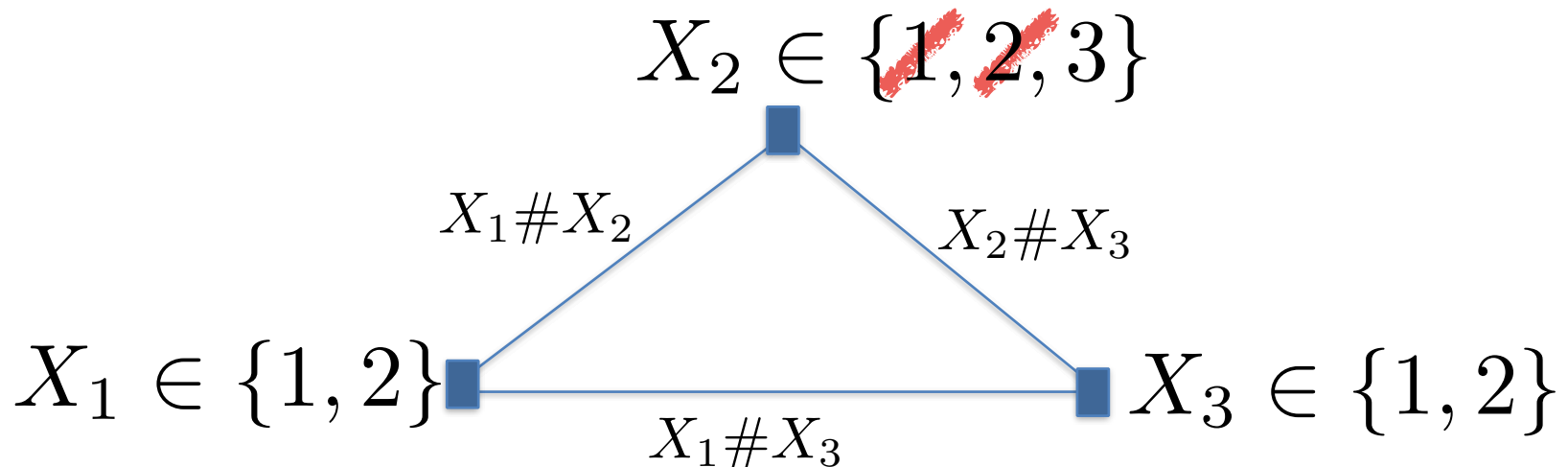
Global Constraints



Global constraint :

- A constraint reasoning on many variables at the same time
- Specialized, powerful filtering

Example : $allDifferent(X_1, X_2, X_3)$



Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

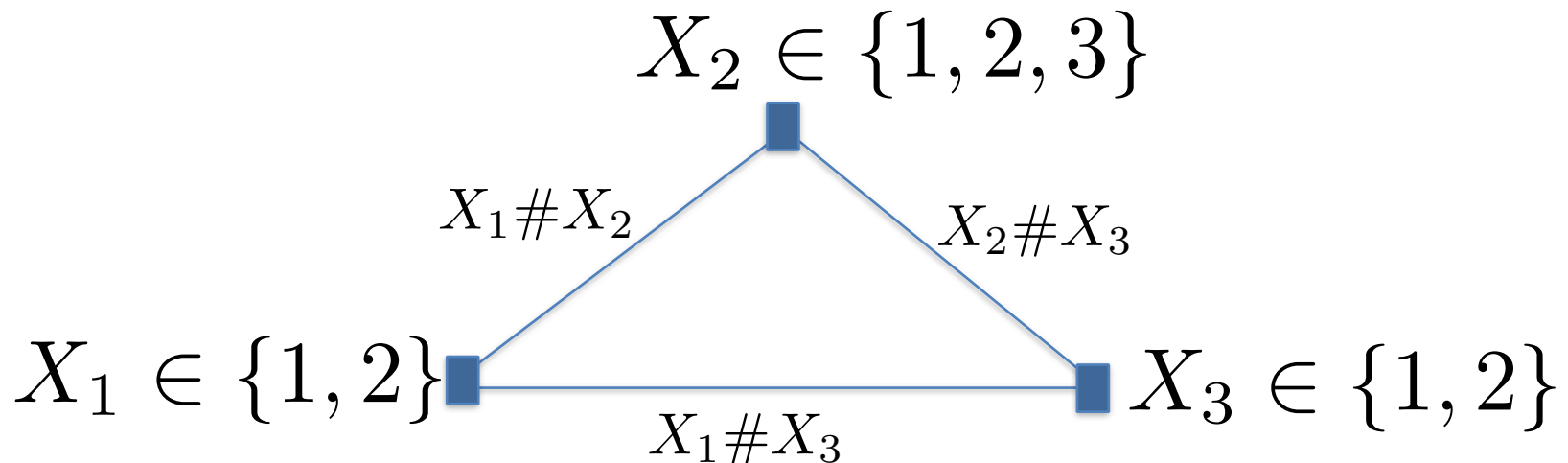
Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Example: *allDifferent*(X_1, X_2, X_3)



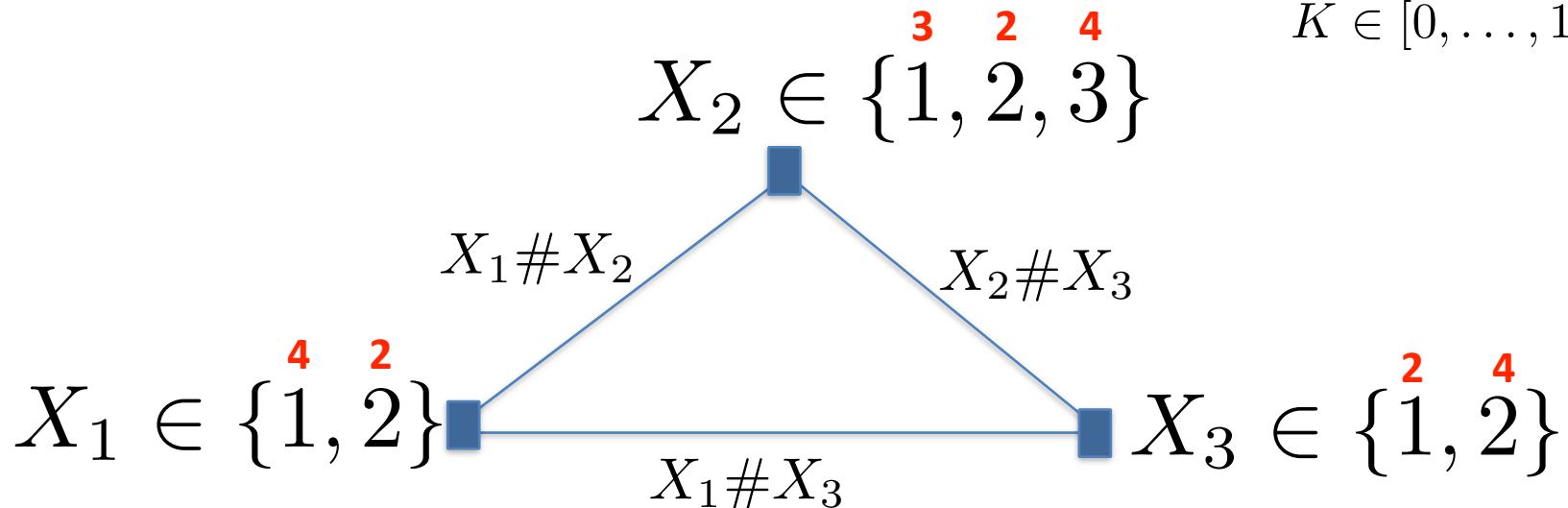
Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Example: $allDifferent(X_1, X_2, X_3)$ with $\sum_i (X_i = j) \cdot cost(X_i \leftarrow j) \leq K$
 $K \in [0, \dots, 10]$



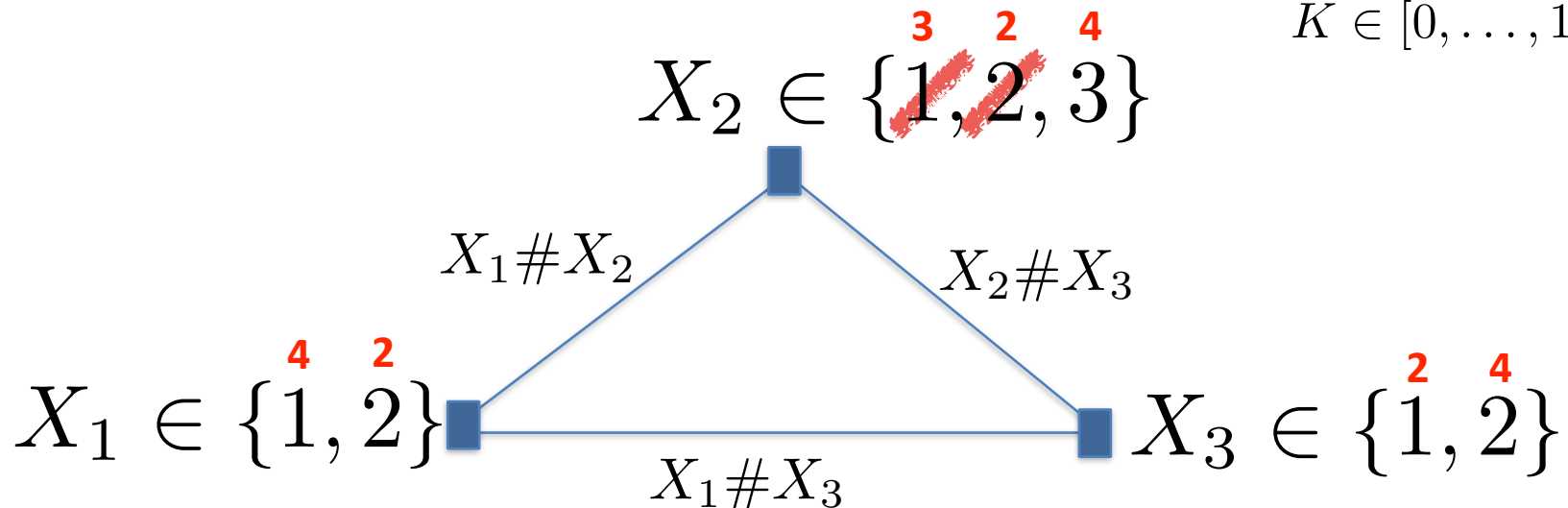
Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Example: $allDifferent(X_1, X_2, X_3)$ with $\sum_i (X_i = j) \cdot cost(X_i \leftarrow j) \leq K$
 $K \in [0, \dots, 10]$



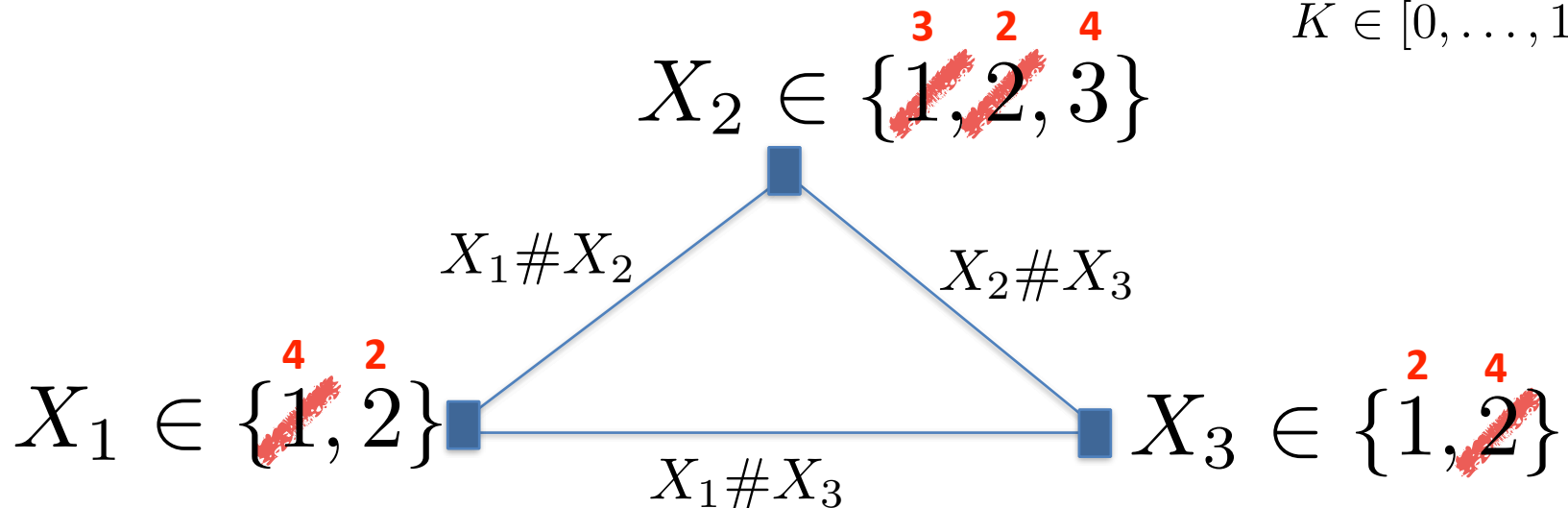
Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Example: $allDifferent(X_1, X_2, X_3)$ with $\sum_i (X_i = j) \cdot cost(X_i \leftarrow j) \leq K$
 $K \in [0, \dots, 10]$



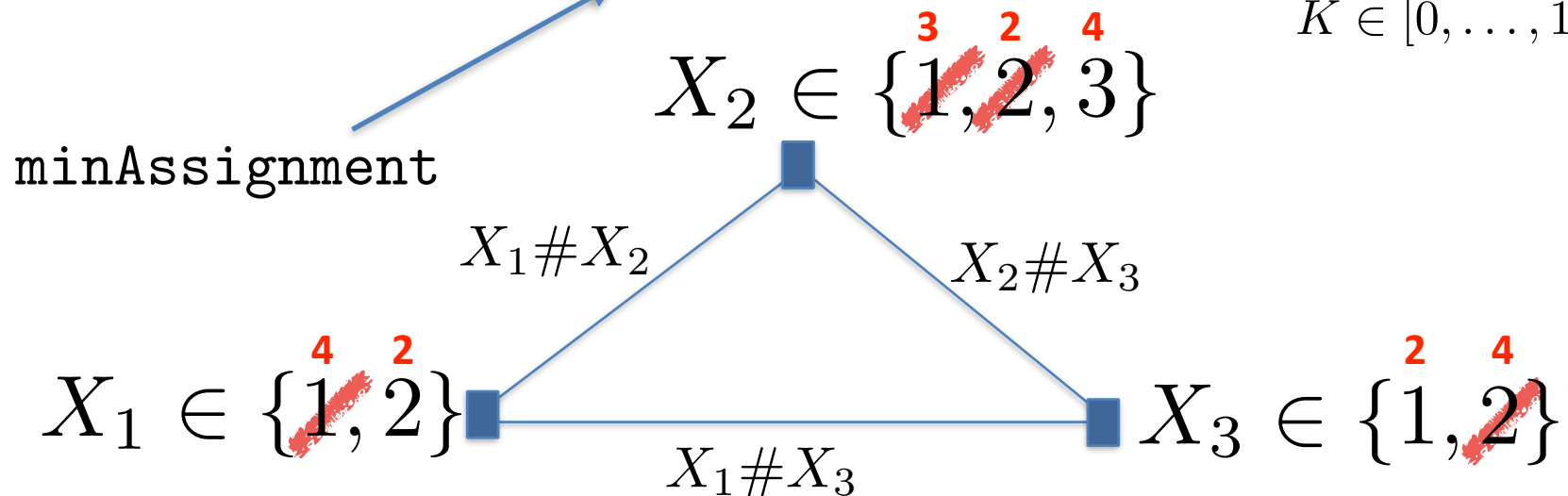
Global Constraints



Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Example: $allDifferent(X_1, X_2, X_3)$ with $\sum_i (X_i = j) \cdot cost(X_i \leftarrow j) \leq K$
 $K \in [0, \dots, 10]$



Global Constraints



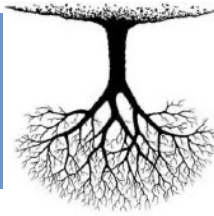
Cost-based filtering

- Filtering for optimization constraint
- Effective for optimization problem

Mainly based on:

- an optimal solution of a related (relaxed) problem
- the optimal value of this solution is used to filter the objective variable
- an optimistic evaluation of the cost increase if a value is assigned to a variable

Global Constraints: Catalog



A catalog of more than 400 global constraints.

<http://sofdem.github.io/gccat/>

5.12. alldifferent

DESCRIPTION LINKS GRAPH AUTOMATON

Origin [\[Lauriere78\]](#)

Constraint `alldifferent(VARIABLES)`

Synonyms `alldiff`, `alldistinct`, `distinct`, `bound _ alldifferent`, `bound _ alldiff`, `bound _ distinct`, `rel`.

Argument `VARIABLES` `collection(var – dvar)`

Restriction `required(VARIABLES, var)`

Purpose Enforce all variables of the collection `VARIABLES` to take distinct values.

Example `((5, 1, 9, 3))`
The `alldifferent` constraint holds since all the values 5, 1, 9 and 3 are distinct.

All solutions Figure 5.12.1 gives all solutions to the following non ground instance of the `alldifferent` constraint: $V_1 \in [2, 4]$, $V_2 \in [2, 3]$, $V_3 \in [1, 6]$, $V_4 \in [2, 5]$, $V_5 \in [2, 3]$, $V_6 \in [1, 6]$, `alldifferent((V1, V2, V3, V4, V5, V6))`.

5. Global Constraint Catalogue

51. `abs_value`
52. `all_differ_from_at_least_k_pos`
53. `all_differ_from_at_most_k_pos`
54. `all_differ_from_exactly_k_pos`
55. `all_equal`
56. `all_equal_peak`
57. `all_equal_peak_max`
58. `all_equal_valley`
59. `all_equal_valley_min`
510. `all_incomparable`
511. `all_min_dist`
512. `alldifferent`
513. `alldifferent_between_sets`
514. `alldifferent_consecutive_values`
515. `alldifferent_cst`
516. `alldifferent_except_0`
517. `alldifferent_interval`
518. `alldifferent_modulo`

This catalog presents a list of 423 global constraints issued from the literature in constraint programming and from popular constraint systems.

Global Constraints



Some examples

allDifferent

globalCardinalityConstraint

minimumAssignment

regular

binPacking

stockingCost

minimumArborescence

Outline

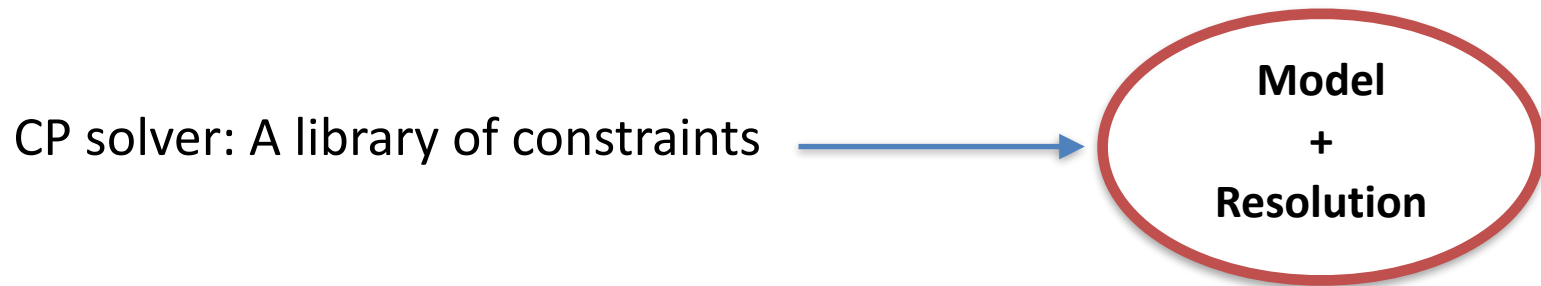
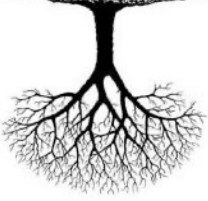
General Introduction

Global Constraints

How CP works concretely ?

Take Away Message

How CP works concretely?



CP Modelling = Putting together the different constraints of the problem.

CP Solvers Catalog



Many CP-solvers exist to ease the implementation of model.

<http://openjvm.jvmhost.net/CPSolvers/>

Constraint Programming Solvers

Product List
Short Catalog
Complete Catalog
Add/Modify Product

This Catalog contains detailed profiles of products submitted and maintained by their developers. The submitters have an exclusive access and are solely responsible for the quality of the provided information about their products. Products are sorted alphabetically. From the Product List you may select products and Compare them feature-by-feature. The Short/Complete Catalog shows All Products allowing business application developers to compare them feature-by-feature. By clicking on the button "Add/Modify Product", authorised people may create profiles of their products and then maintain them in the up-to-date state.

[List of Available Product Profiles](#)

Brief lists of [Constraint Programming](#) tools may be found at these sources: www.cspib.org, www.constraint-solving.com, Wikipedia [Constraint Solvers](#).

Select up to 3 Products and click on [Compare Selected Products](#)

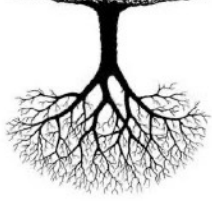
Sponsors

#	Product	Select
1	AIMMS	☐
2	AMPL	☐
3	Cardinal	☐
4	CasPER	☐
5	CHIP	☐
6	Choco	☐
7	Chuffed	☐
8	Concorate	☐
9	Copris	☐
10	Cream	☐
11	ECLIPSe	☐
12	Geocode	☐
13	HaifaCSP	☐
14	IBM CSP Solver	☐
15	IBM ILOG CP Optimizer	☐
16	Intelligem	☐
17	IZ-C	☐
18	JeCoP	☐

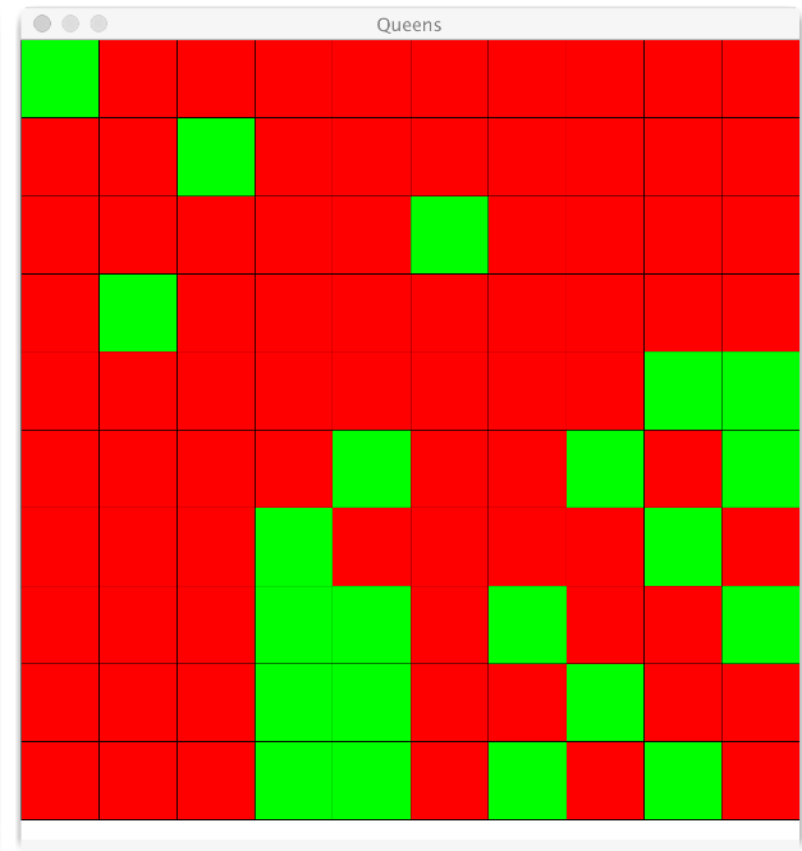
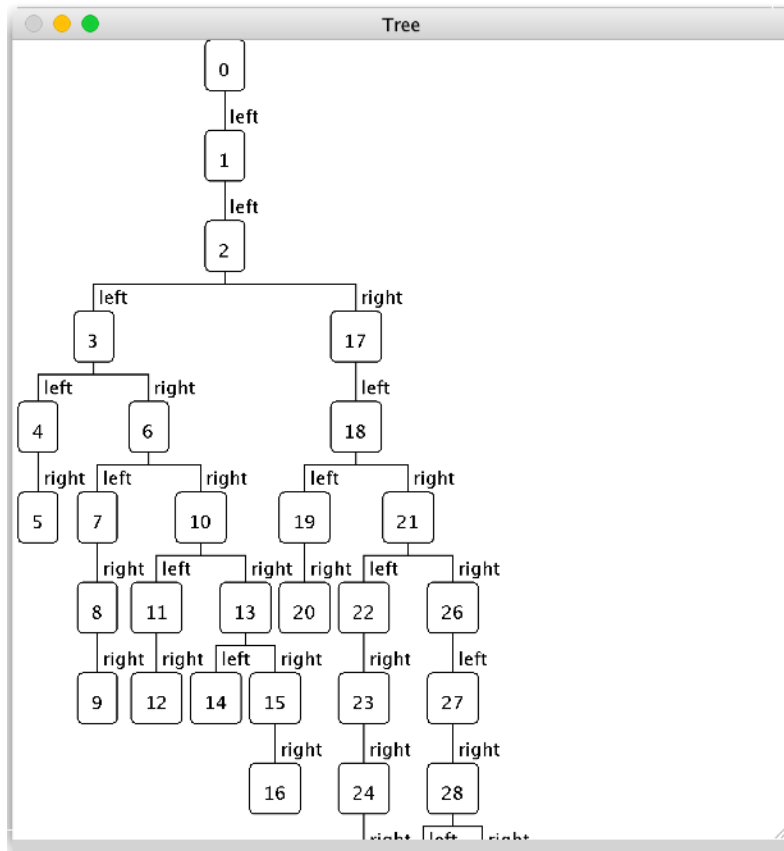
ACP
OPEN

This catalog presents a list of 35 CP based solvers (August 13, 2018).

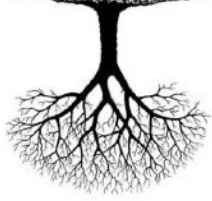
How CP works concretely?



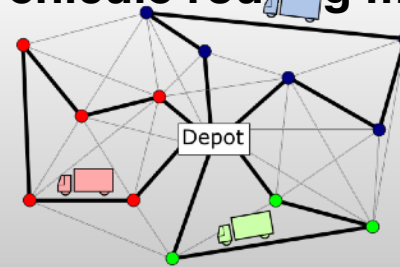
A live demo with the solver Oscar.



How CP works concretely?

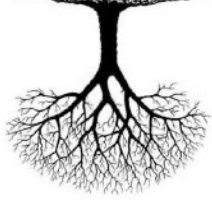


A vehicle routing model



Put together the different constraints.

The job of a constraint !



- Given the status of the domain, do you think that you still have a solution?



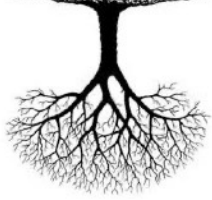
- Remove the values from the domains that you think are impossible in a solution (= **filtering or pruning**)



A constraint in action!

Impossible values

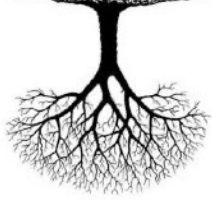
variables and
domains



allDifferent: A stronger Filtering Algorithm

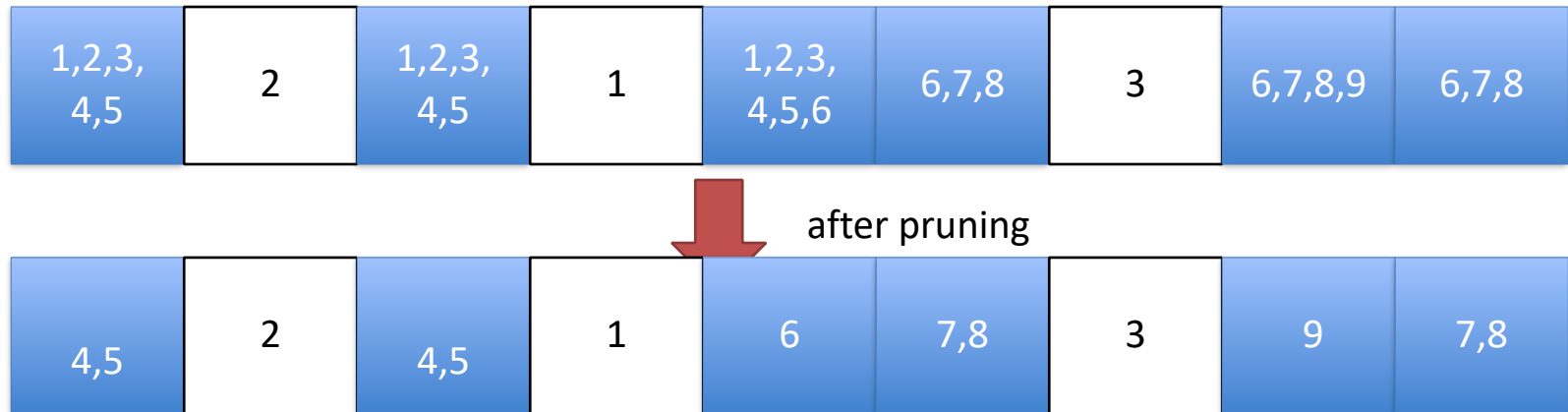
- Remove any impossible values

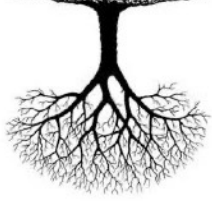
x1= {1,2,3, 4,5}	x2={2}	x3= {1,2,3, 4,5}	x4= {1}	x5= {1,2,3, 4,5,6}	x6= {6,7,8}	x7= {3}	x8= {6,7,8,9}	x9= {6,7,8}
------------------------	--------	------------------------	---------	--------------------------	----------------	---------	------------------	----------------



allDifferent: A stronger Filtering Algorithm

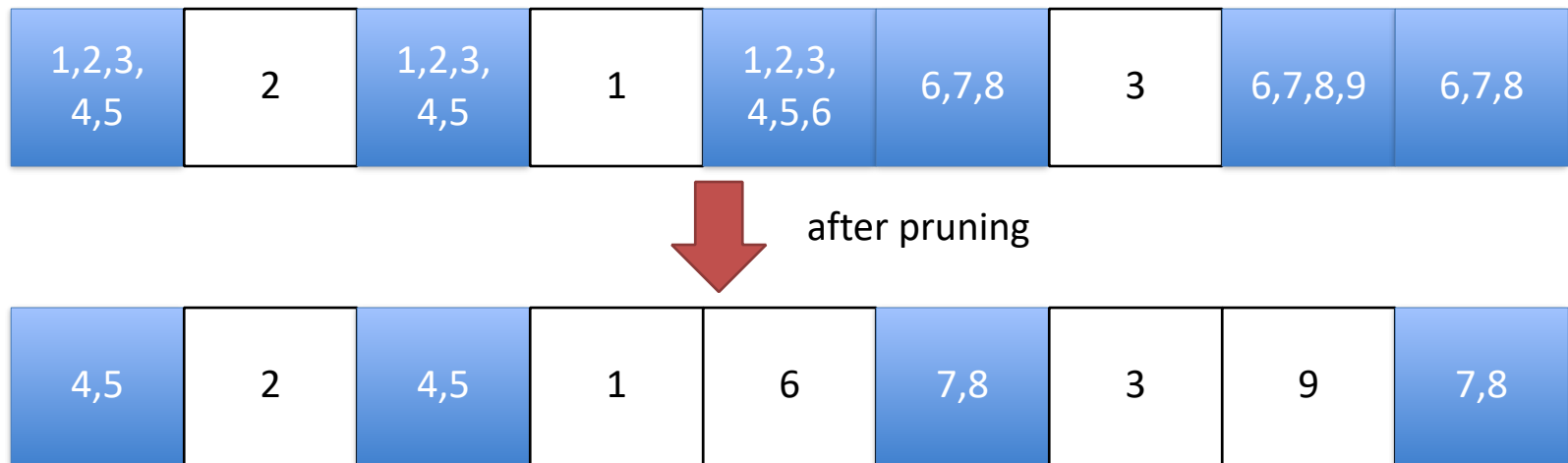
- Remove any impossible values





allDifferent: A stronger Filtering Algorithm

- Remove any impossible values



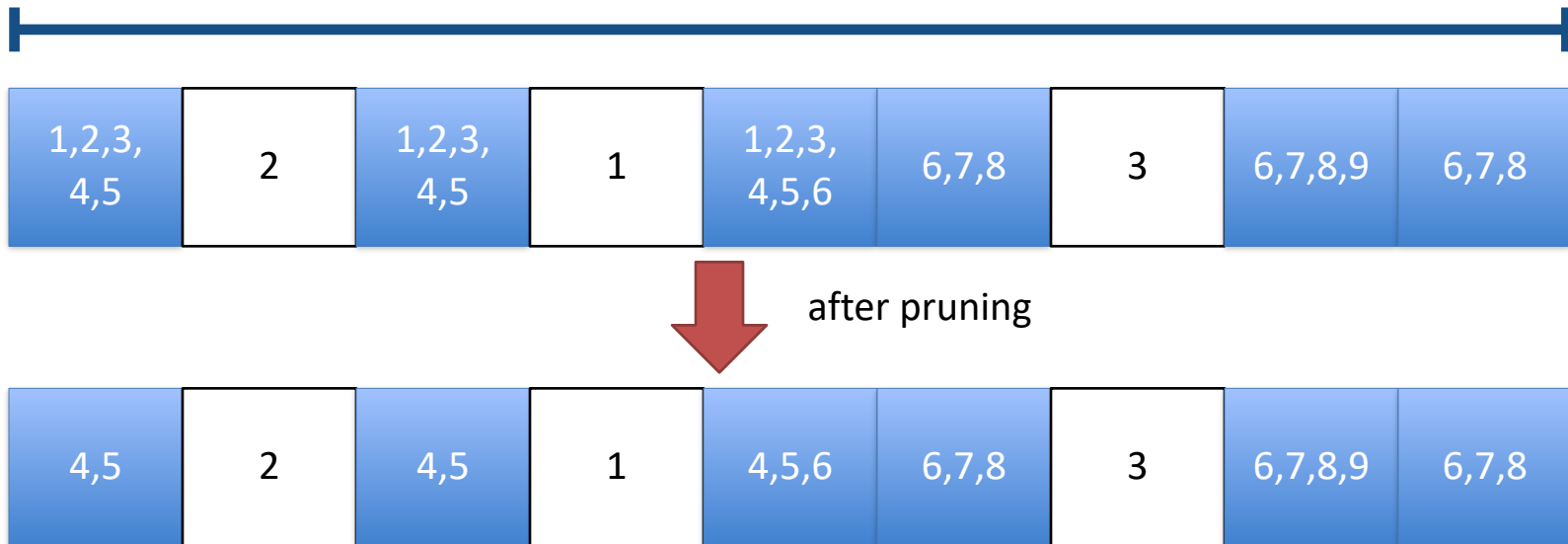
→ A more global reasoning



allDifferent: Basic (fast) Filtering Algorithm

- As soon as a variable x_i has a single value v
Remove v from the domain of every other variable

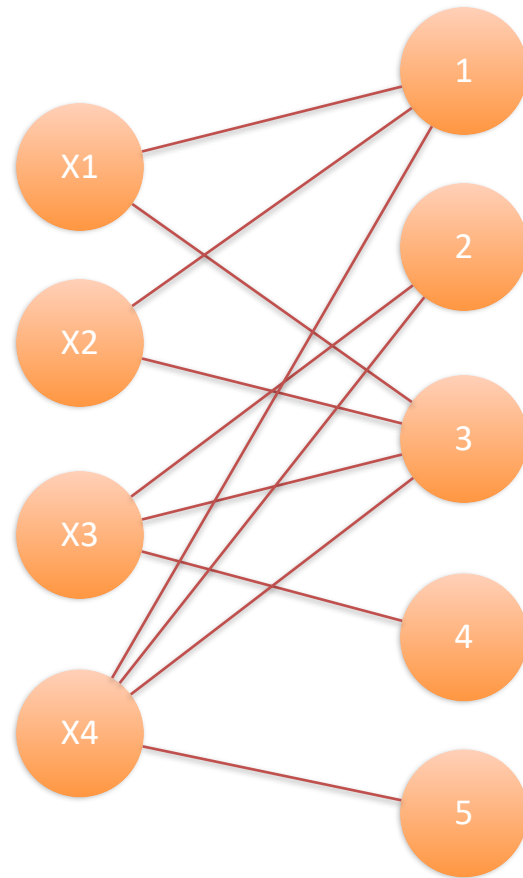
allDifferent



Weak pruning for allDifferent is fast and easy to implement but does not remove all inconsistent values

Global Filtering for allDifferent (J-C. Régin Algorithm, 95)

- Build the variable-value graph



Global Filtering for allDifferent (J-C. Régin Algorithm, 95)

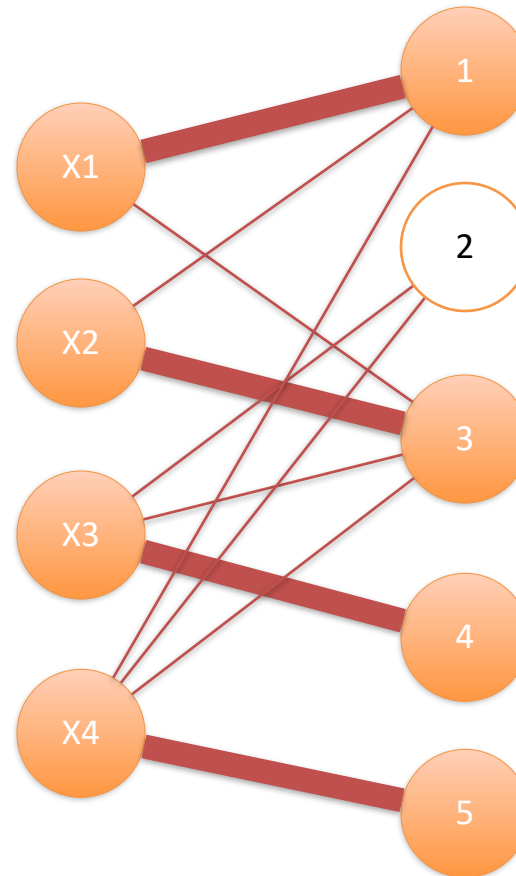
Consistency Check:

- There is a solution to $\text{allDifferent}([X1, X2, X3, X4])$ iff there is a maximum matching M (of size 4) in the variable value graph

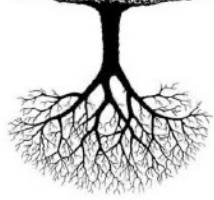
Filtering:

- Remove edges that do not belong to some maximum matching

Requires to compute Strongly connected components in residual graph

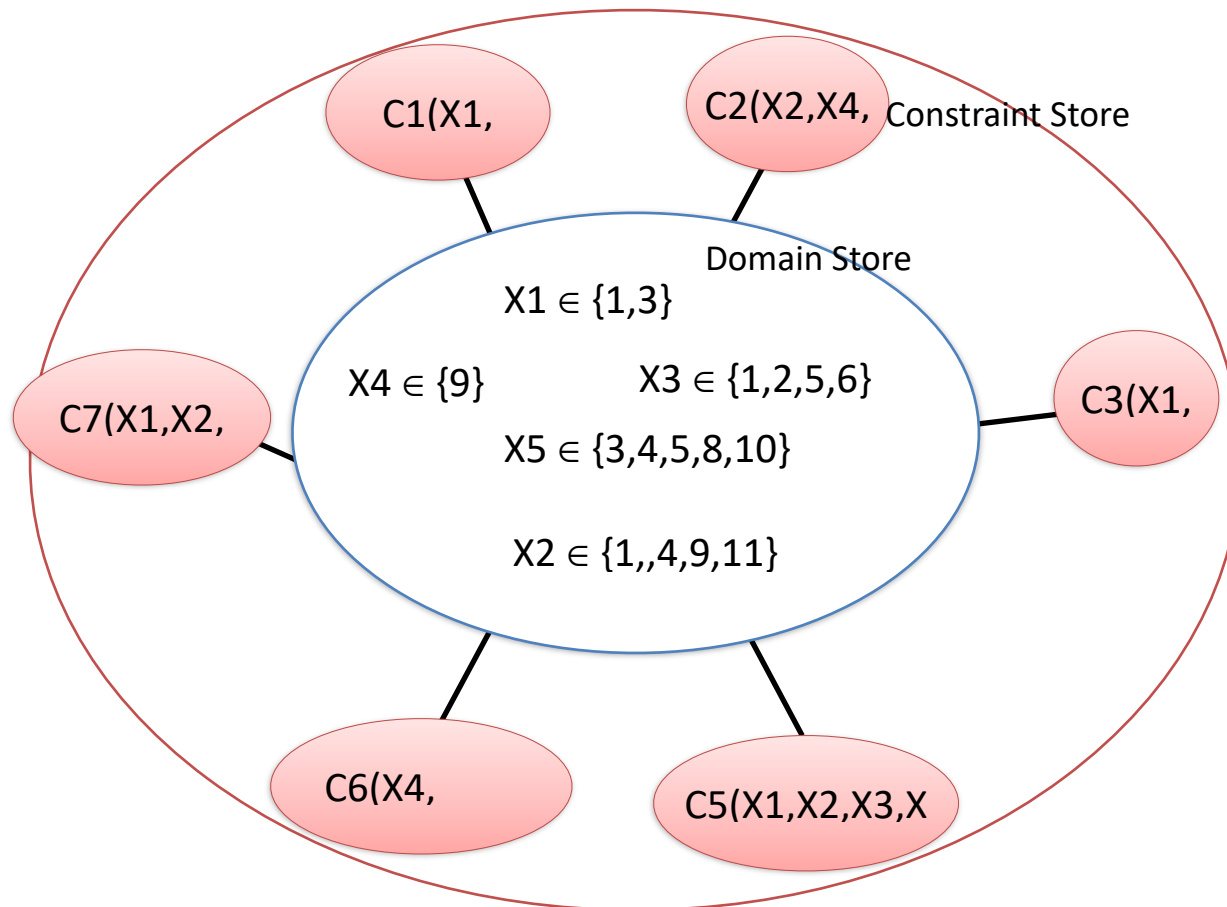


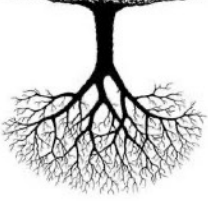
How CP works concretely?



Each constraint must remove impossible values.

Yes but the pruning of one constraint might trigger new deductions for other constraints...





Fix-Point Algorithm

```
repeat
  select a constraint c
  if c is OK wrt the domain store
    apply filtering algorithm of c
  else
    return KO
until no value can be removed
```

Back to our gold seeker illustration

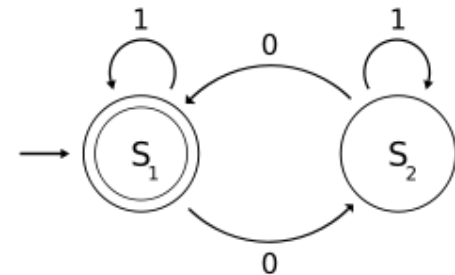
Fix-point Algo

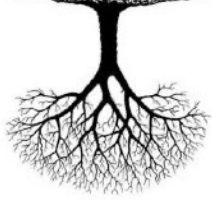


sum($X[]$): $\sum_i X(i) = 0$

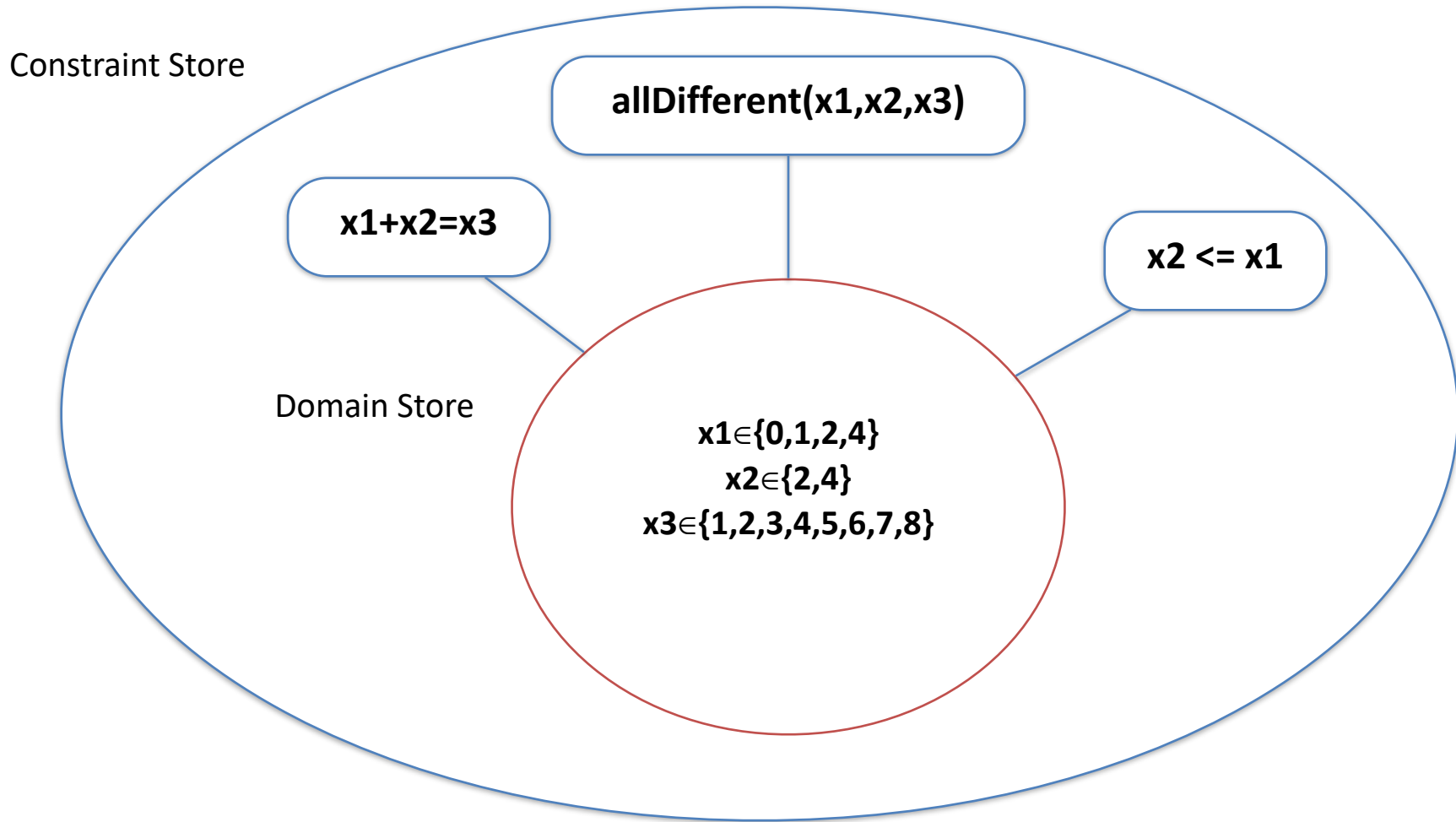
allDifferent($X[]$): $\forall i < j : X(i) \neq X(j)$

regular($X[]$):

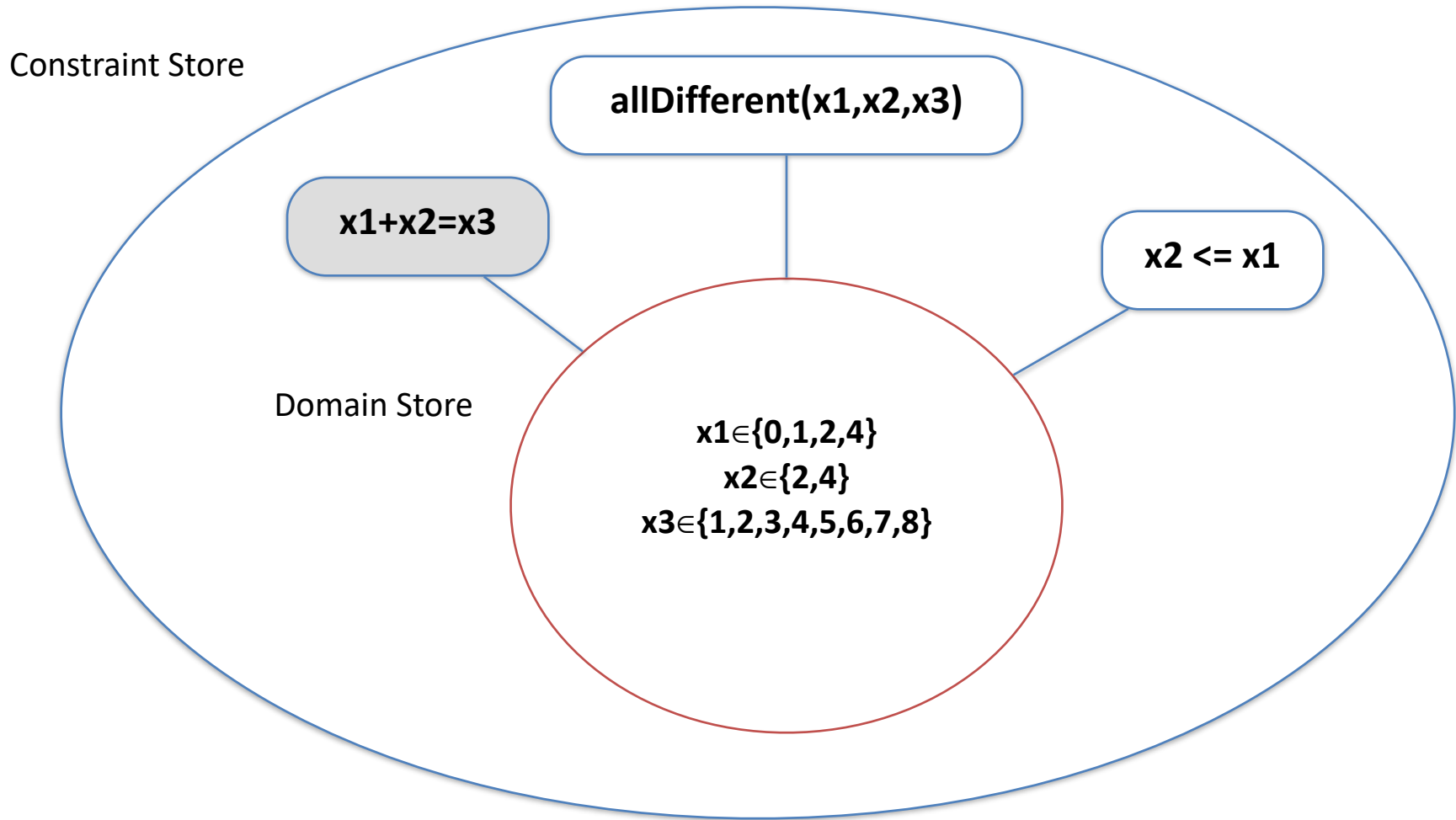




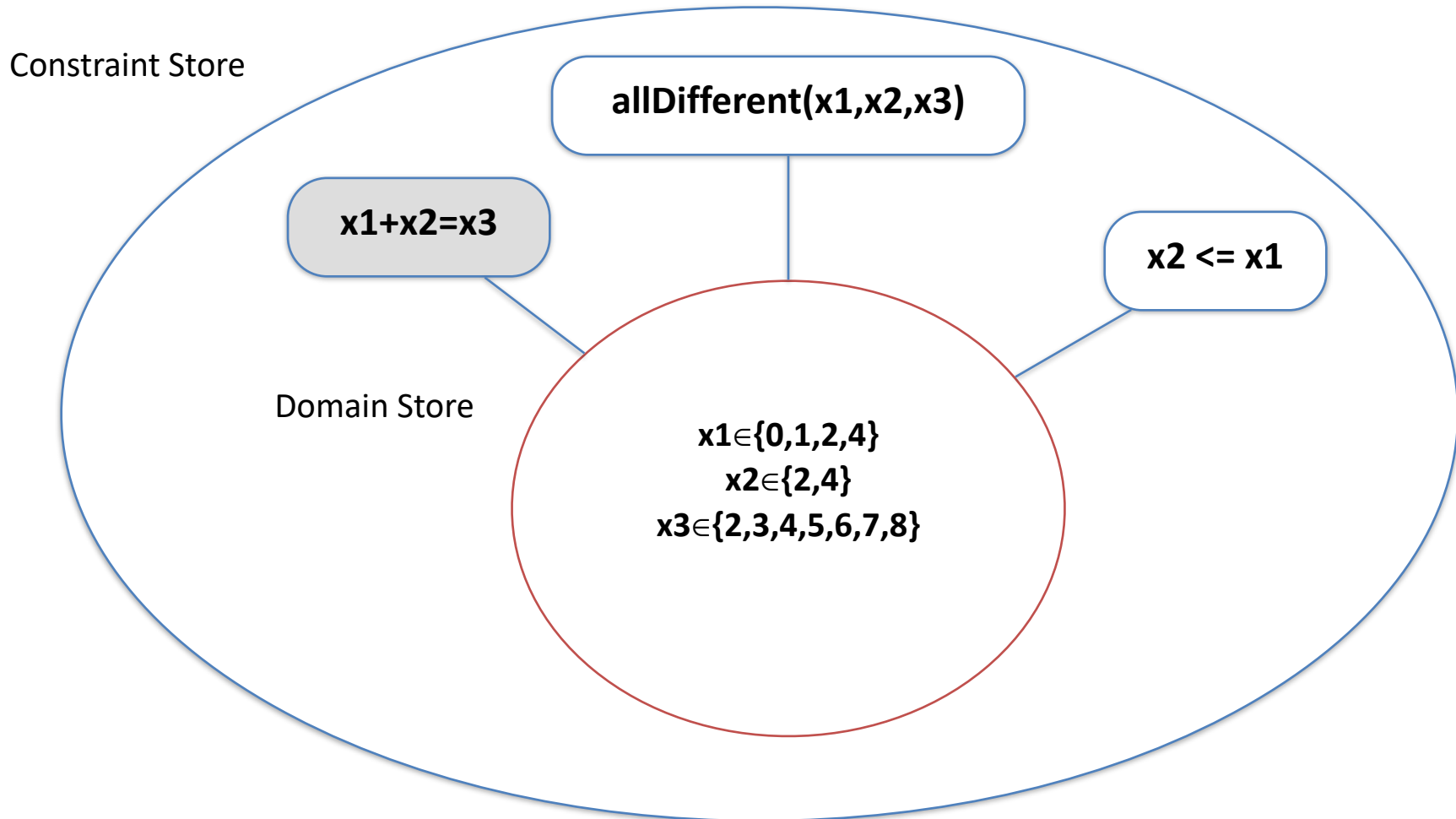
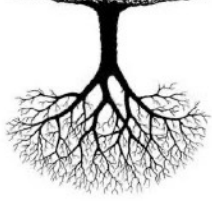
Fix-Point Example



How CP works concretely?

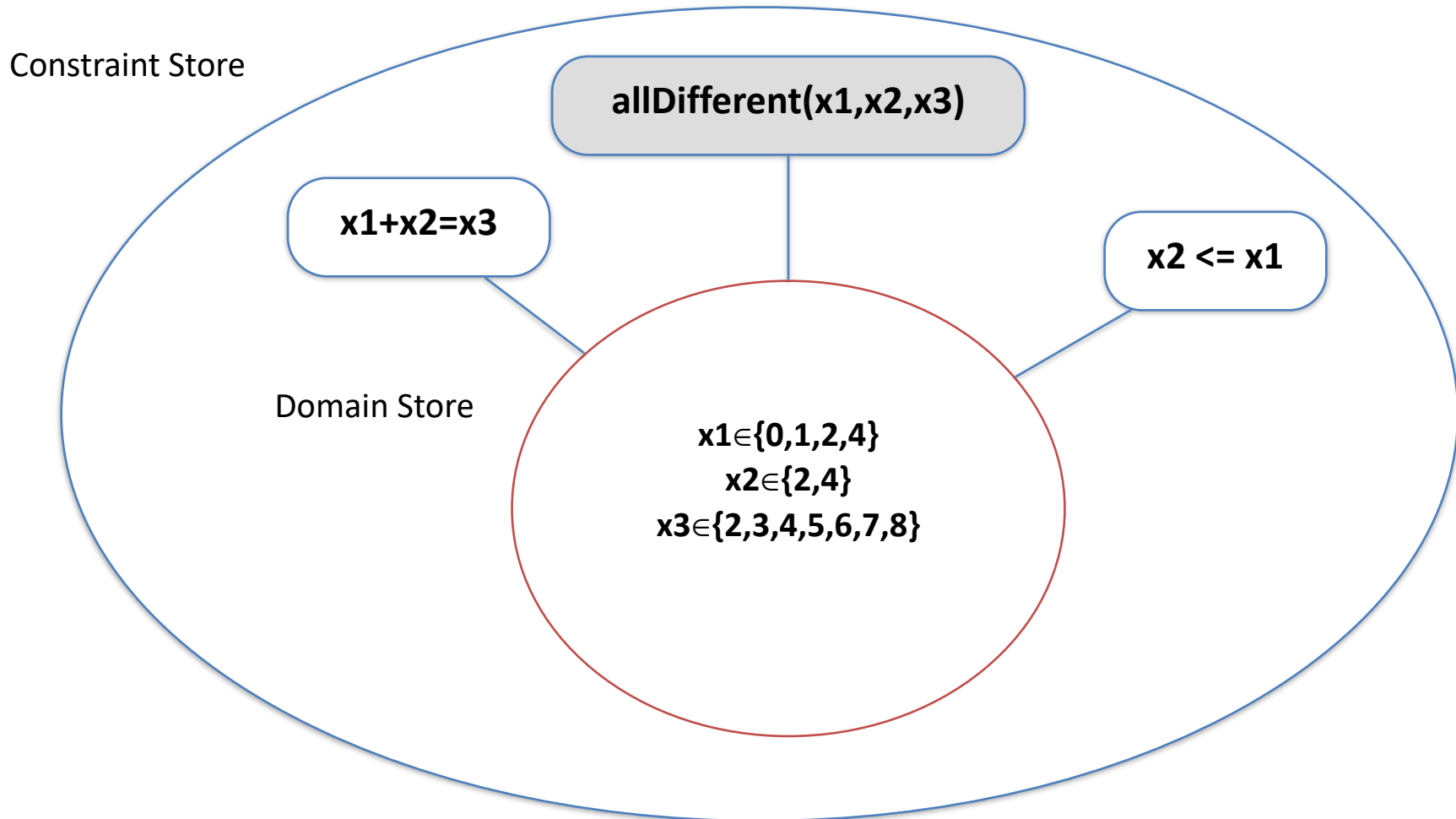
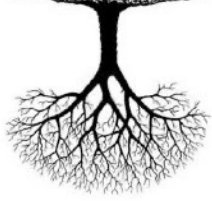


How CP works concretely?



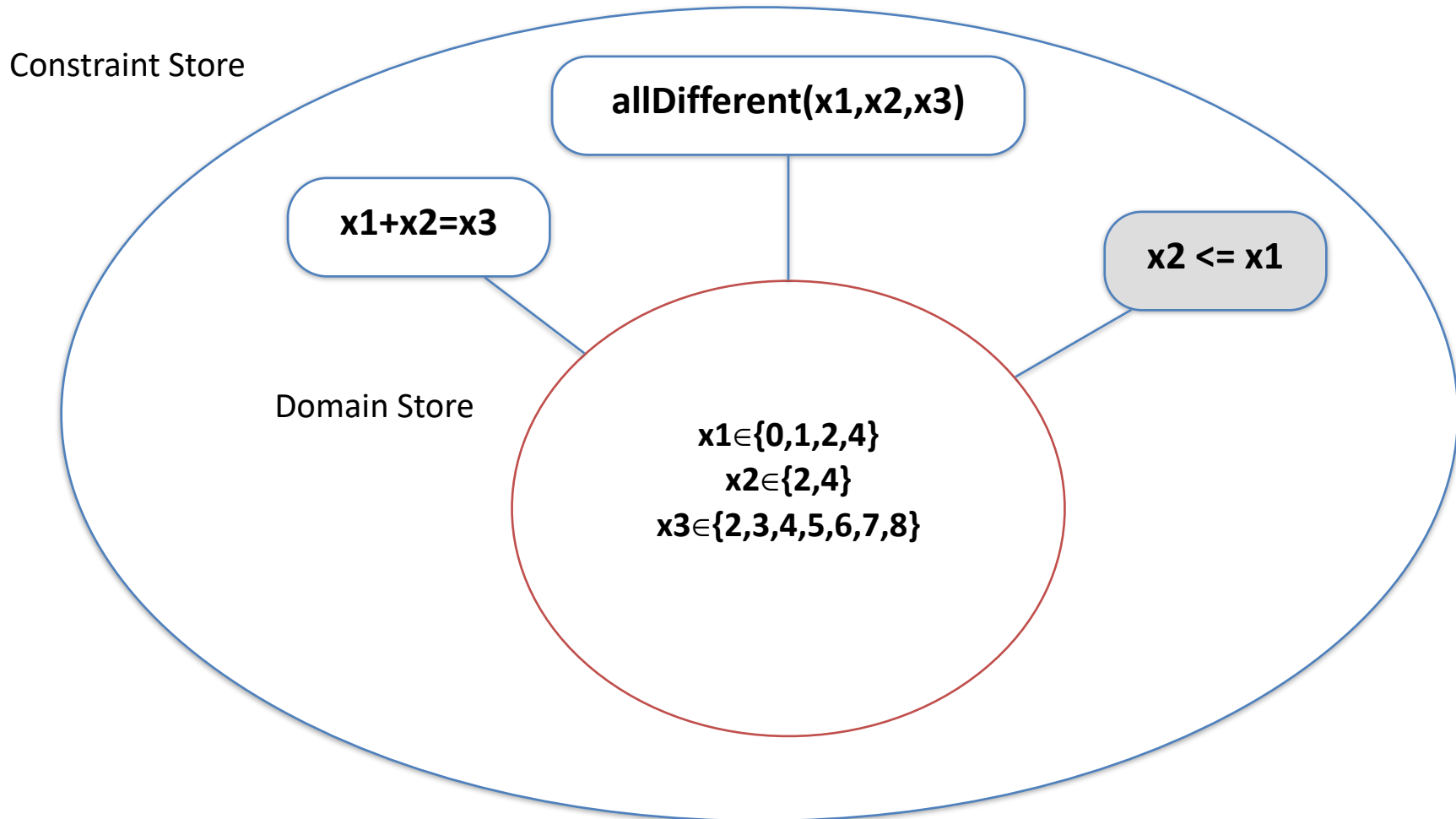
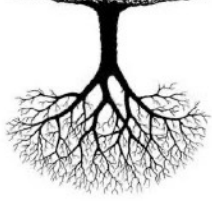
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



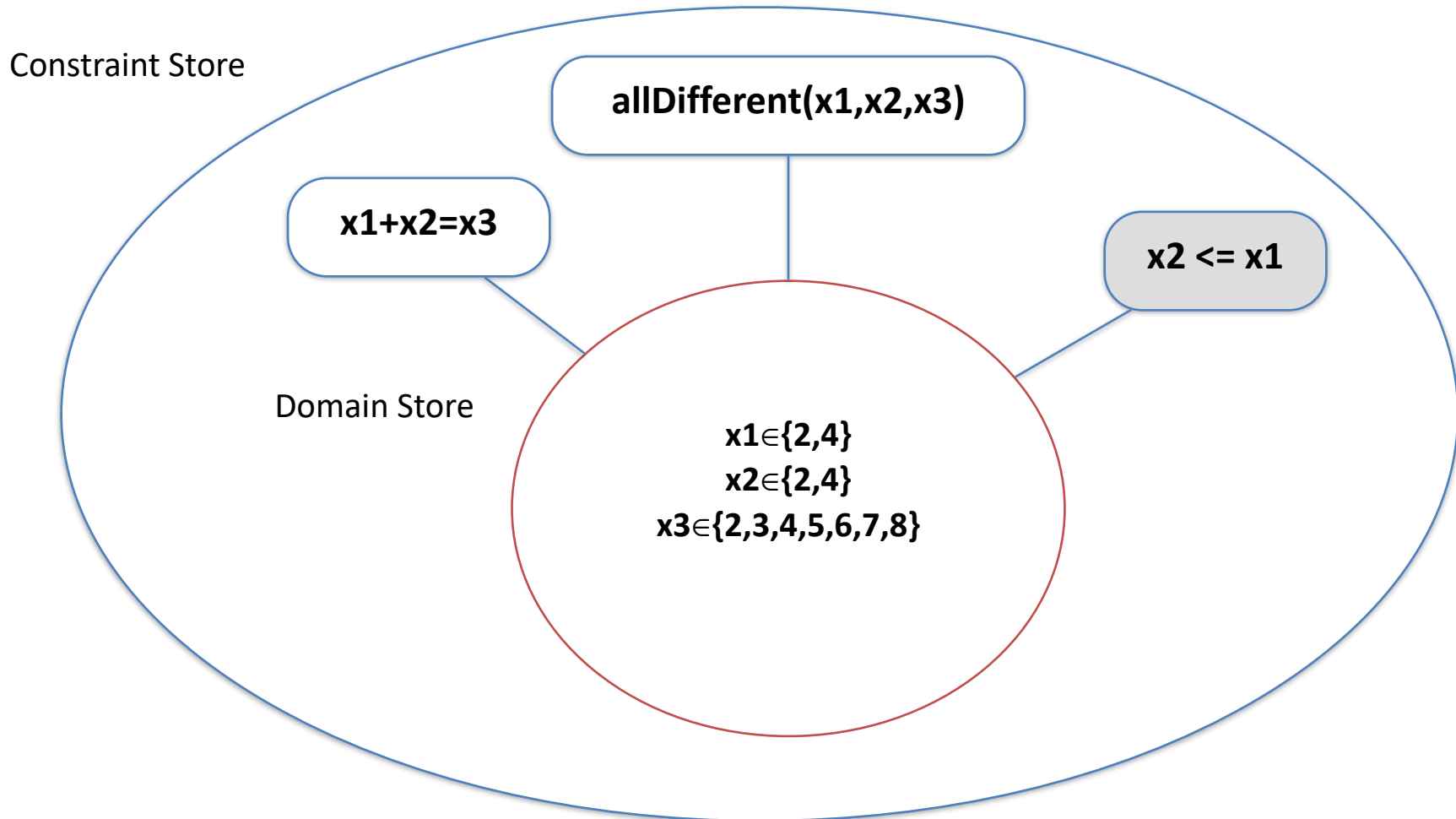
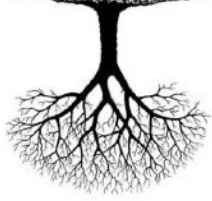
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



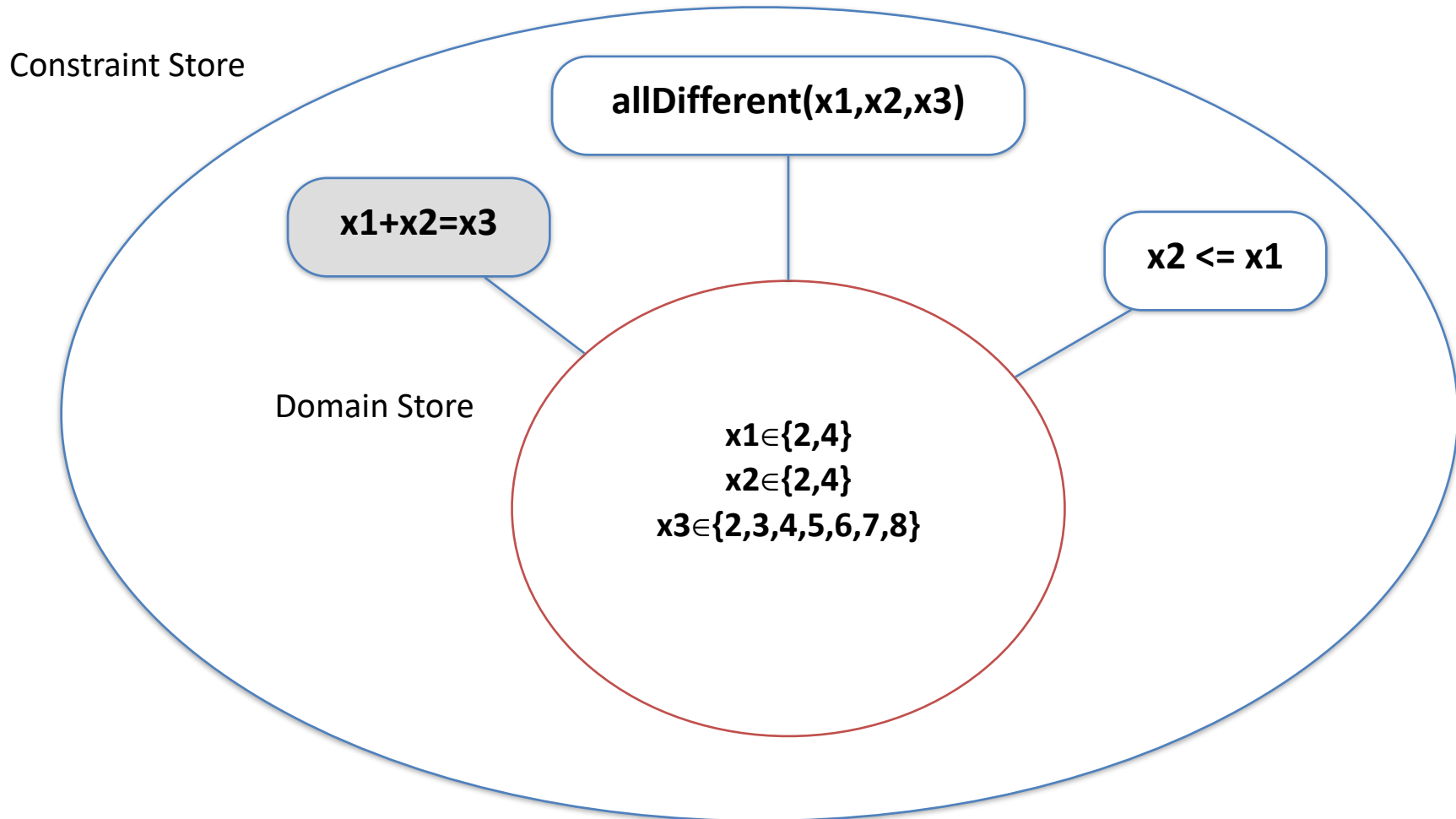
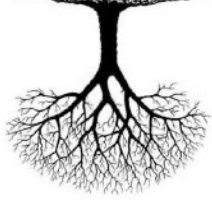
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



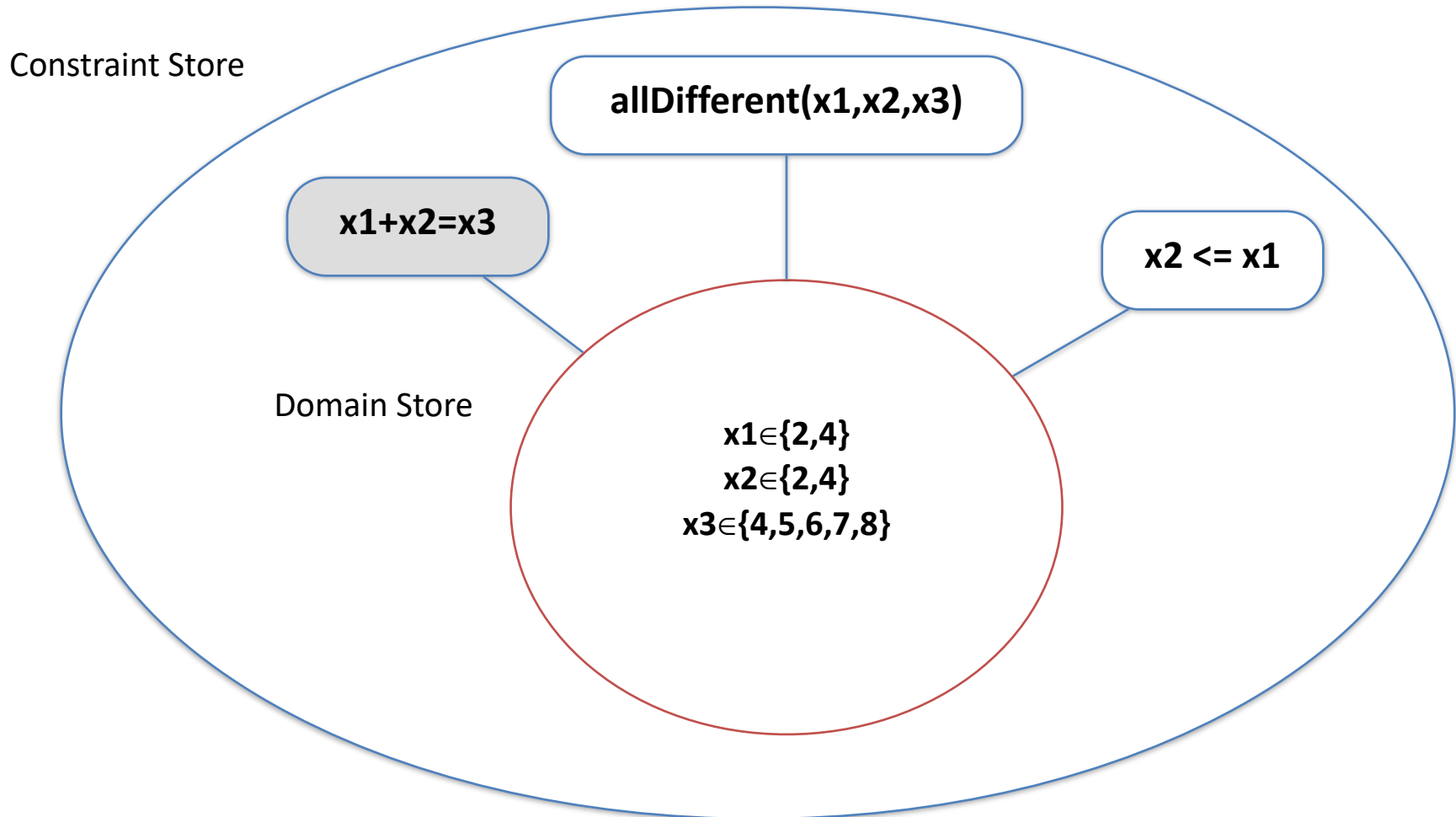
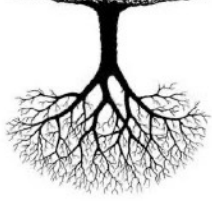
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



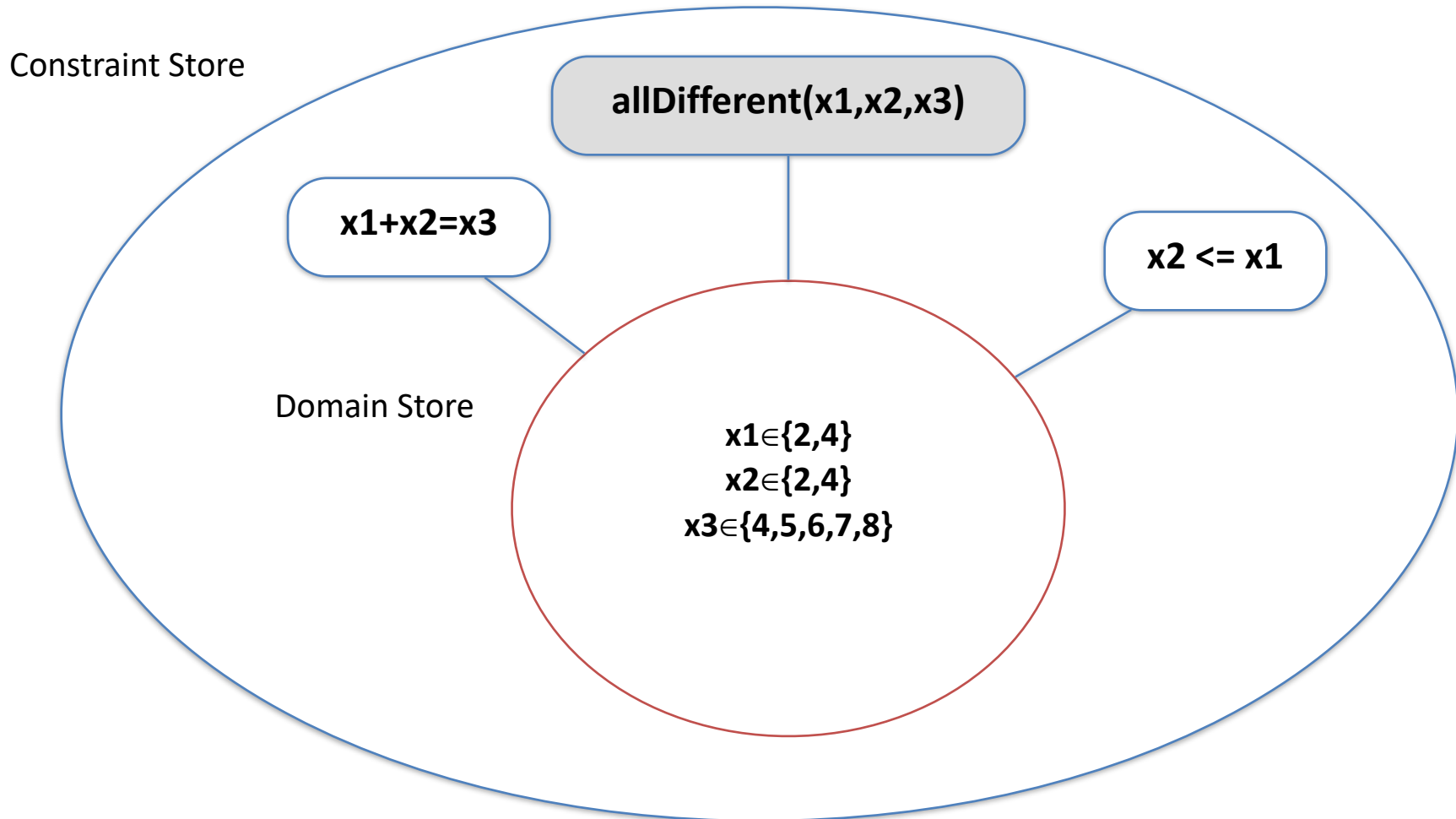
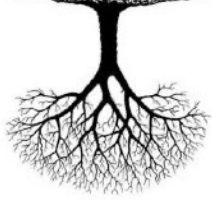
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



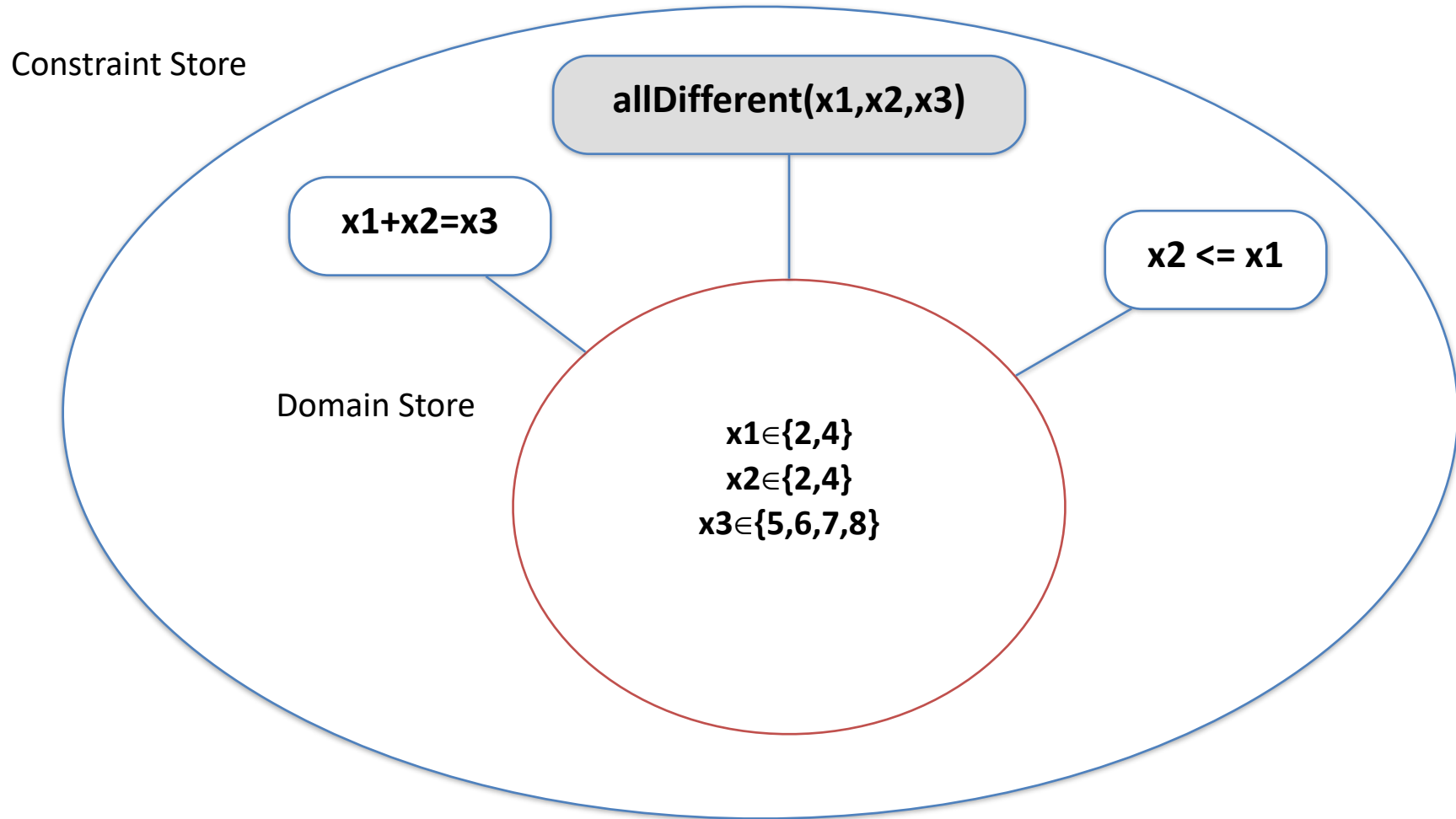
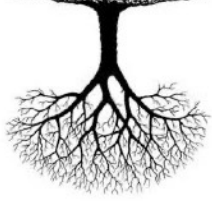
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



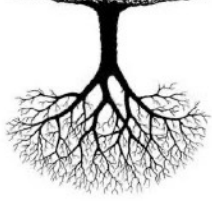
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?



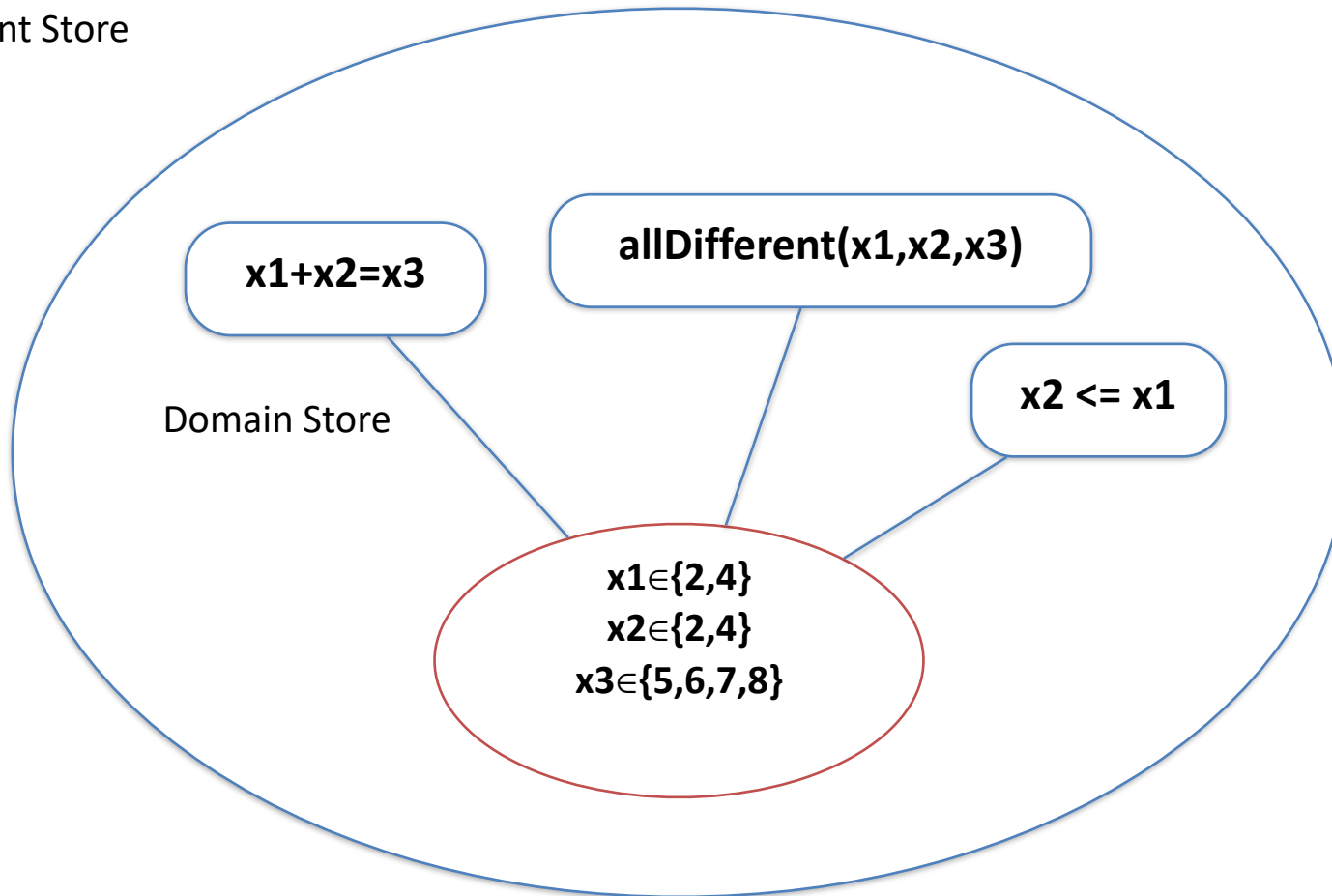
ps: We assume the sum constraint only reasons on bounds of the domains

How CP works concretely?

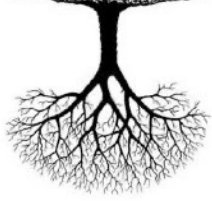


Fix-Point! but not a solution yet ...
We need to search ...

Constraint Store



ps: We assume the sum constraint only reasons on bounds of the domains



Model Example: n-Queens in OscalaR CP

```
object Queens extends App with CPModel {
```

```
  val n = 12 //number of queens
```

```
  val Queens = 0 until n
```

```
  //variables
```

```
  val queens = for (i <- Queens) yield CPIntVar(1 to n)
```

```
  onSolution { nSol += 1 }
```

```
  add(allDifferent(queens))
```

```
  add(allDifferent(for (i <- Queens) yield queens(i) + i))
```

```
  add(allDifferent(for (i <- Queens) yield queens(i) - i))
```

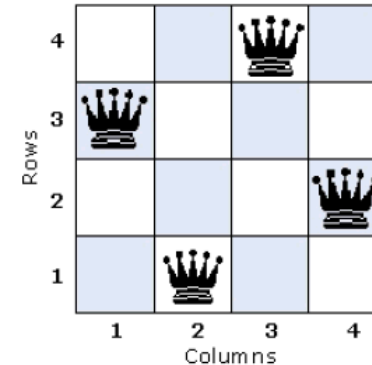
```
  search {
```

```
    binaryFirstFail(queens)
```

```
  }
```

```
  start()
```

```
}
```



variables and
domains

constraints

default search

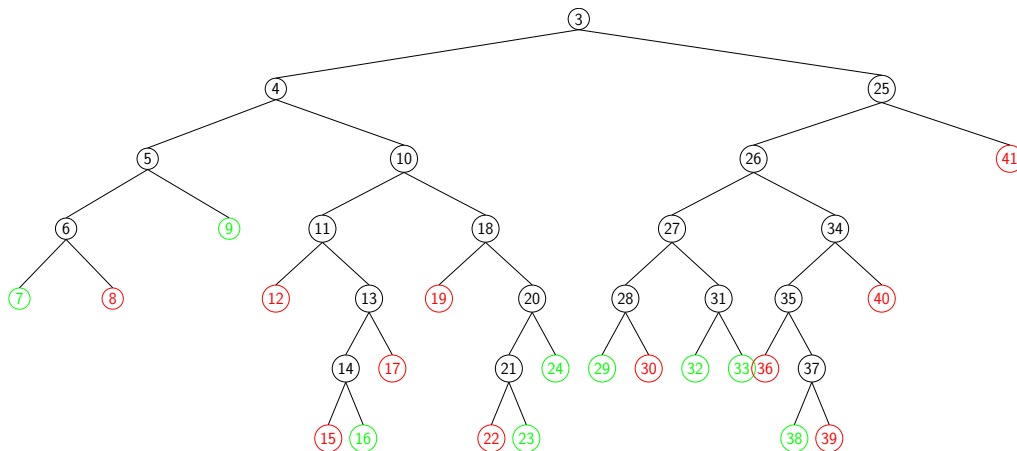
How CP works concretely?



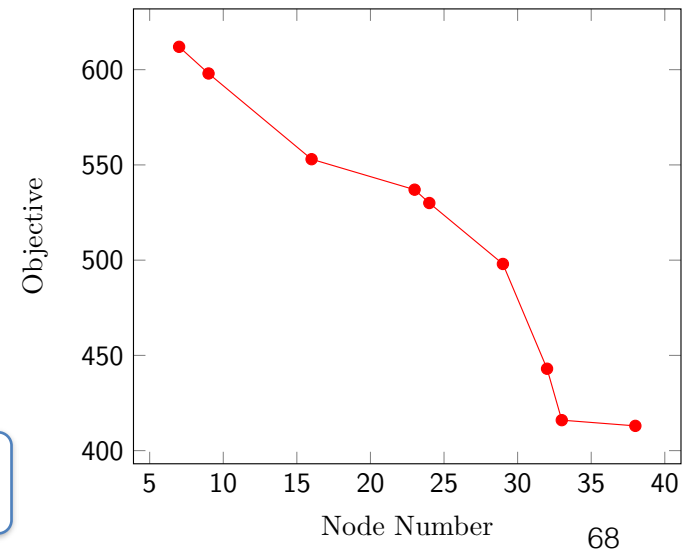
Optimization with CP

```
// your constraints  
cp.minimize(objVar)  
} search {  
  branching(decVars)  
} start()
```

```
// your constraints  
} search {  
  branching(decVars)  
} onSolution {  
  updateBound()  
} start()
```

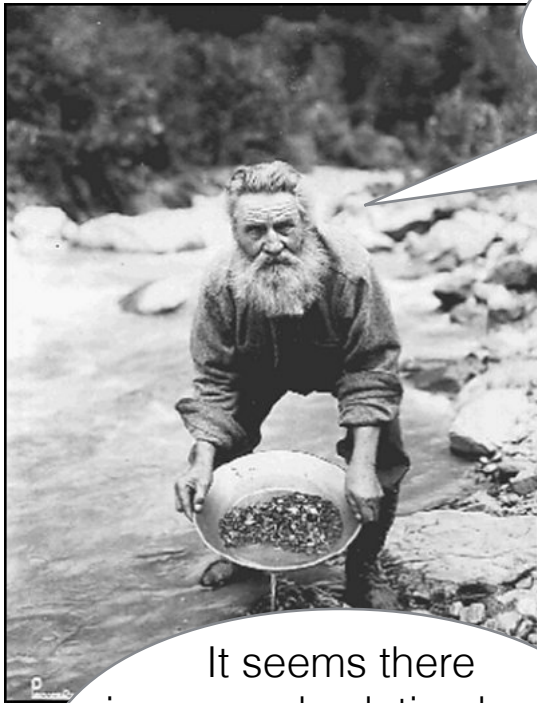


Best solution



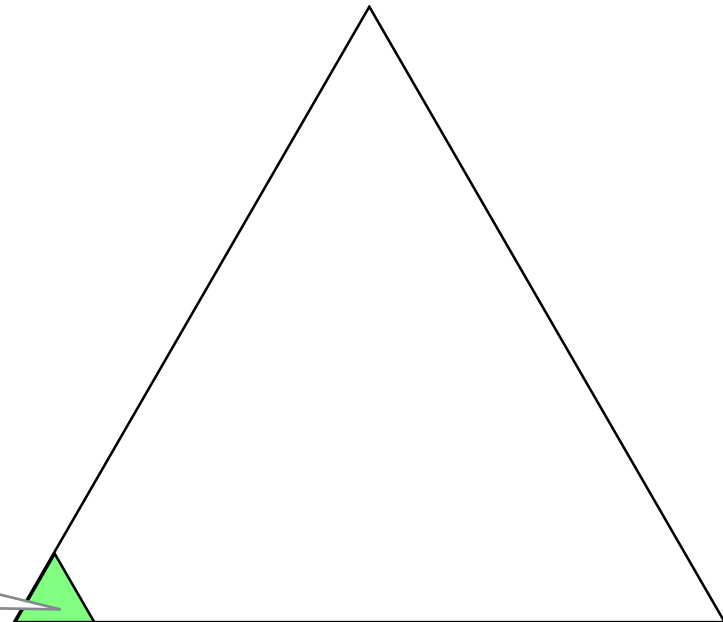


Large Neighborhood Search



Search Algo: « No gold here!
I should not wait too long before moving at
another place »

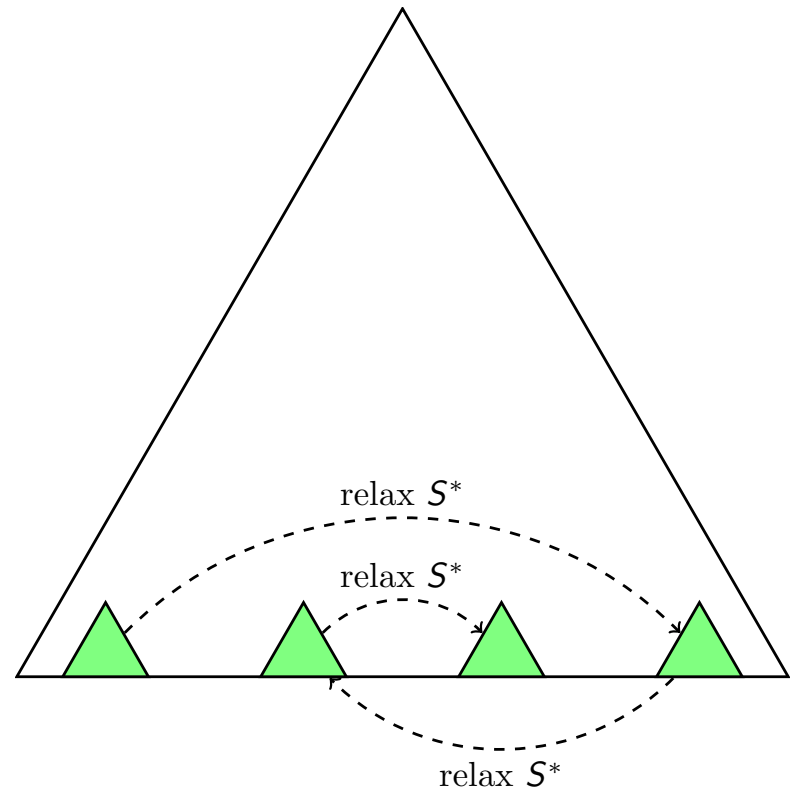
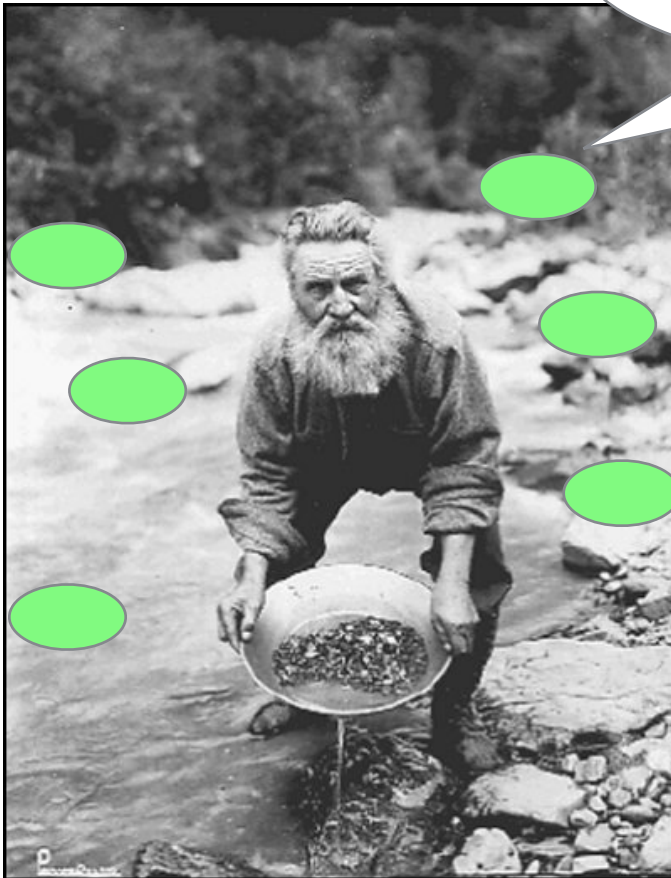
It seems there
is no good solution here!
I should not wait too long
before moving at another
place





Large Neighborhood Search

Search Algo: « I change my strategy,
I'll visit randomly other places for limited
amount of time »



Outline

General Introduction

Global Constraints

How CP works concretely ?

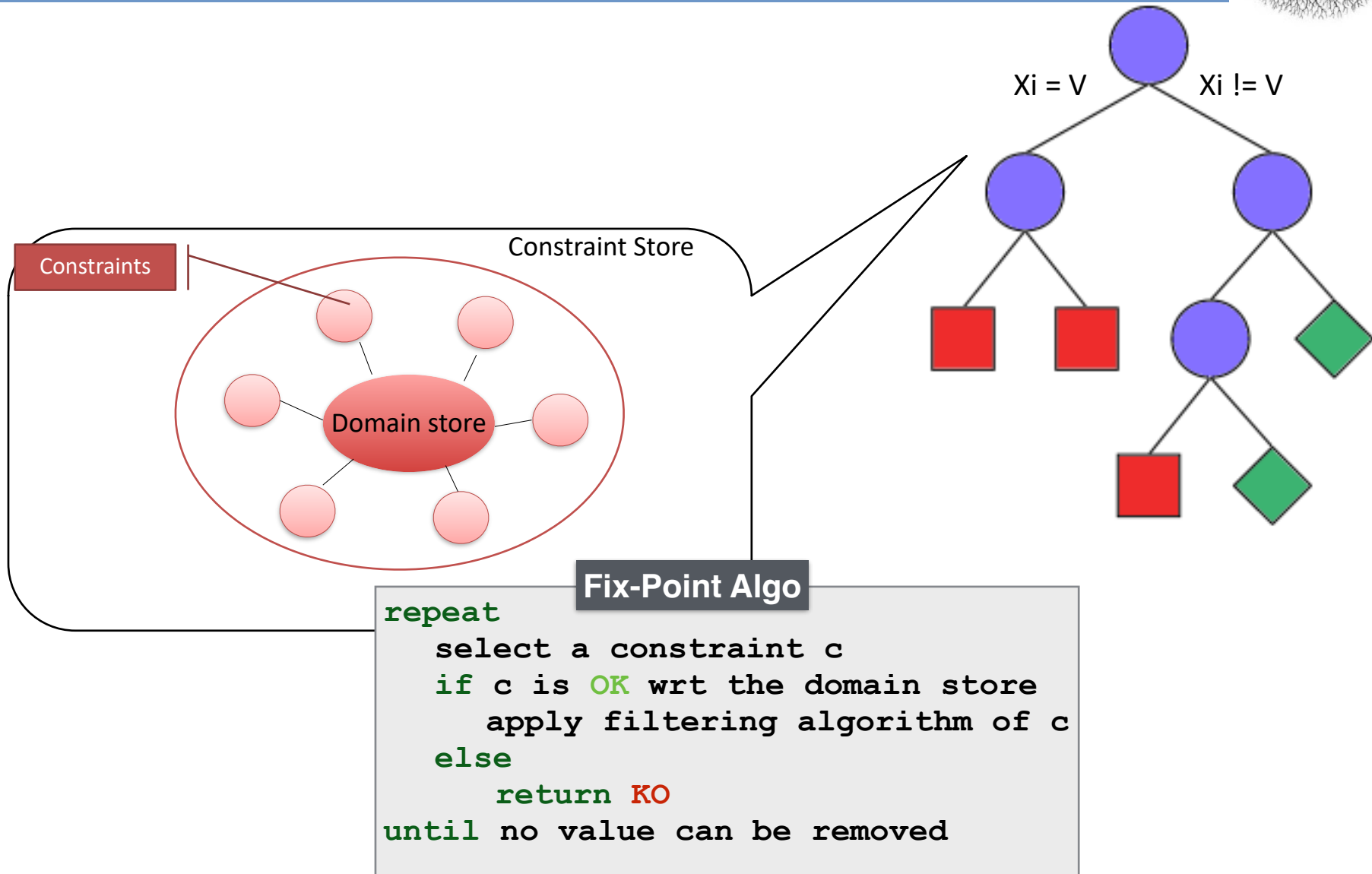
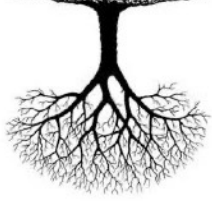
Take Away Message

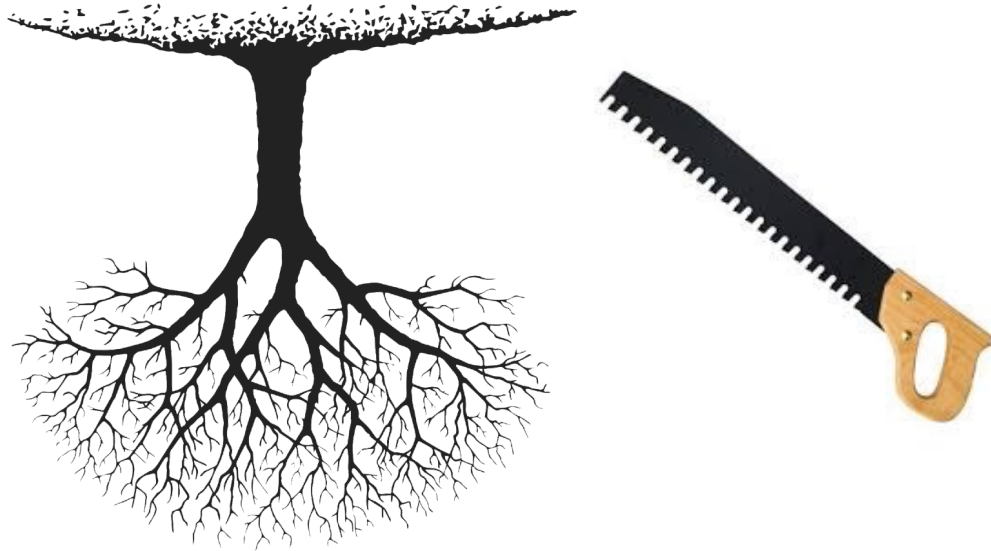


You must experiment CP !!



Take away message





Thank you for your attention

Some references

Pierre Schaus, Constraint Programming: A tool inspired by gold seekers. Grascomp, 2015. [Some slides are from this reference]

CSPLib: A problem library for constraints. csplib.org

A catalog of global constraints. <http://sofdem.github.io/gccat/>

CP Solvers Catalog. <http://openjvm.jvmhost.net/CPSolvers/>

Mini-CP: A lightweight Constraint Programming Solver <http://www.minicp.org/>

Oscar is a Scala toolkit for solving Operations Research problems.
<https://bitbucket.org/oscarlib/oscar/wiki/Home>